

LAPORAN TUGAS
IF3270 Machine Learning

**Implementasi CNN, RNN, dan LSTM untuk Image Classification
dan Image Captioning**



Disusun oleh :

Bryan Ho	13523029
Alvin Christopher Santausa	13523033
Kenneth Poenadi	13523040

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2026

BAB 1

Deskripsi Persoalan

A. Penjelasan Singkat

Deep learning merupakan salah satu pendekatan dalam *machine learning* yang banyak digunakan untuk menyelesaikan permasalahan berbasis data kompleks, seperti gambar dan teks. Dalam tugas ini, persoalan utama yang dibahas adalah bagaimana mengimplementasikan dan menganalisis dua jenis arsitektur deep learning, yaitu Convolutional Neural Network (CNN) untuk klasifikasi gambar serta Recurrent Neural Network (RNN) dan Long Short-Term Memory (LSTM) untuk image captioning.

Pada bagian pertama, CNN digunakan untuk menyelesaikan task image classification menggunakan dataset Intel Image Classification yang terdiri dari sekitar 25.000 gambar dengan 6 kategori, yaitu buildings, forest, glacier, mountain, sea, dan street. Persoalan utama pada bagian ini adalah bagaimana membangun model CNN yang mampu mengenali pola visual dari gambar melalui proses konvolusi, pooling, flattening, dan dense layer. Selain itu, tugas ini juga mencakup implementasi forward propagation from scratch menggunakan NumPy, sehingga proses internal CNN seperti operasi Conv2D, LocallyConnected2D, pooling, aktivasi, dan dense layer dapat dipahami secara lebih mendalam. Model kemudian dianalisis berdasarkan variasi hyperparameter seperti jumlah layer konvolusi, jumlah filter, ukuran filter, dan jenis pooling layer.

Pada bagian kedua, RNN dan LSTM digunakan untuk menyelesaikan task image captioning menggunakan dataset Flickr8k, yang terdiri dari 8.092 gambar dengan 5 caption untuk setiap gambar. Persoalan utama pada bagian ini adalah bagaimana menghasilkan deskripsi teks berdasarkan input gambar. CNN digunakan sebagai encoder untuk mengekstraksi fitur visual dari gambar, sedangkan RNN dan LSTM digunakan sebagai decoder untuk menghasilkan urutan kata dalam bentuk caption. Implementasi dilakukan dengan mengacu pada arsitektur Show and Tell, di mana feature vector dari CNN dimasukkan ke dalam decoder sebelum token awal caption. Kedua decoder kemudian dibandingkan berdasarkan kualitas caption, waktu eksekusi, serta metrik evaluasi seperti BLEU-4 dan METEOR score.

B. CNN

Convolutional Neural Network (CNN) merupakan salah satu arsitektur deep learning yang dikembangkan khusus untuk mengolah data berbentuk gambar. CNN bekerja dengan cara meniru kemampuan manusia dalam mengenali pola visual, seperti garis, warna, bentuk, tekstur, hingga objek tertentu. Dengan kata lain, CNN mampu mempelajari ciri-ciri penting dari sebuah gambar secara bertahap, mulai dari fitur sederhana hingga fitur yang lebih kompleks.

CNN terdiri dari beberapa lapisan utama, yaitu Convolutional Layer, Pooling Layer, Flatten atau Global Pooling Layer, dan Dense Layer. Convolutional Layer berfungsi untuk mengekstraksi fitur dari gambar menggunakan filter atau kernel yang bergerak pada area gambar. Setiap filter akan menangkap pola tertentu, seperti tepi, sudut, atau tekstur. Setelah itu, Pooling Layer digunakan

untuk mengurangi ukuran feature map sehingga proses komputasi menjadi lebih efisien dan model lebih fokus pada fitur yang penting.

Setelah fitur gambar berhasil diekstraksi, hasilnya akan diubah menjadi bentuk vektor melalui Flatten atau Global Pooling Layer. Vektor tersebut kemudian diproses oleh Dense Layer untuk menghasilkan prediksi kelas gambar. Dalam tugas ini, CNN digunakan untuk melakukan klasifikasi gambar menggunakan dataset Intel Image Classification yang terdiri dari sekitar 25.000 gambar dengan 6 kategori, yaitu buildings, forest, glacier, mountain, sea, dan street.

Selama proses training, CNN mempelajari bobot filter yang optimal agar dapat mengenali pola visual dengan lebih baik. Proses pembelajaran dilakukan dengan meminimalkan fungsi loss menggunakan optimizer seperti Adam. Dengan kemampuan tersebut, CNN banyak digunakan untuk berbagai tugas pengolahan citra, seperti klasifikasi gambar, deteksi objek, pengenalan wajah, dan analisis gambar medis.

C. RNN / LSTM

Recurrent Neural Network (RNN) merupakan salah satu arsitektur dari Artificial Neural Network (ANN) yang dirancang khusus untuk memproses data berbentuk urutan atau sequence. Berbeda dengan Feedforward Neural Network (FFNN) yang hanya mengalirkan informasi secara satu arah tanpa mengingat proses sebelumnya, RNN memiliki koneksi berulang yang memungkinkan model menyimpan informasi dari timestep sebelumnya. Artinya, output pada suatu tahap tidak hanya dipengaruhi oleh input saat ini, tetapi juga oleh informasi dari proses sebelumnya.

Dalam RNN, setiap input pada urutan data diproses secara bertahap. Pada setiap timestep, neuron menerima input saat ini dan menggabungkannya dengan hidden state dari timestep sebelumnya. Hidden state ini berfungsi sebagai memori sementara yang menyimpan informasi penting dari urutan sebelumnya. Hasil pemrosesan tersebut kemudian digunakan untuk menghasilkan output dan hidden state baru yang akan diteruskan ke timestep berikutnya.

Selama proses training, RNN menyesuaikan bobot dan bias dengan meminimalkan fungsi loss menggunakan algoritma optimasi seperti gradient descent. Perhitungan gradien dilakukan melalui proses backpropagation through time (BPTT), yaitu bentuk backpropagation yang diterapkan pada data berurutan. Namun, RNN memiliki keterbatasan dalam mempelajari hubungan jangka panjang karena dapat mengalami masalah vanishing gradient, yaitu kondisi ketika informasi dari timestep yang jauh semakin melemah selama proses training.

Untuk mengatasi keterbatasan tersebut, digunakan Long Short-Term Memory (LSTM) sebagai pengembangan dari RNN. LSTM memiliki mekanisme gate yang membantu model menentukan informasi mana yang perlu disimpan, dilupakan, atau digunakan dalam prediksi. Dalam tugas ini,

RNN dan LSTM digunakan sebagai decoder untuk task image captioning pada dataset Flickr8k, di mana CNN berperan sebagai mengekstraksi fitur gambar, sedangkan RNN atau LSTM menghasilkan caption secara berurutan.

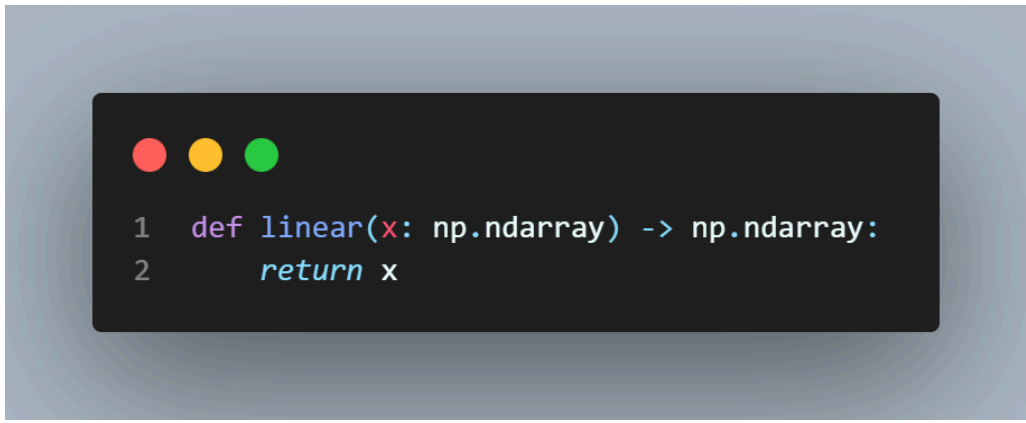
BAB 2

Pembahasan

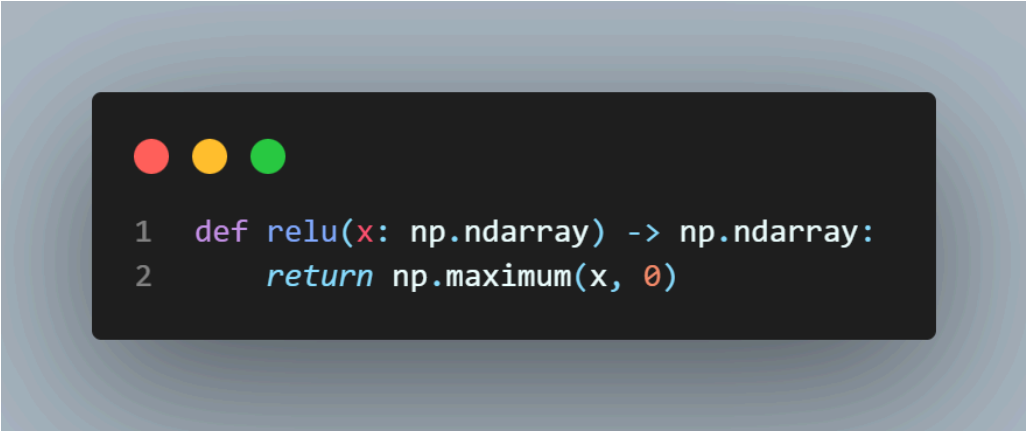
A. Penjelasan Implementasi

Penjelasan Kelas dan Atribut

1. Activation Functions (src/tubes2_ml/scratch/layers/activations.py)

Nama Fungsi	Tipe	Parameter
linear	Function	x: np.ndarray
Deskripsi: Menerapkan fungsi aktivasi linear $f(x) = x$. Fungsi ini mengembalikan input tanpa perubahan.		
 <pre>1 def linear(x: np.ndarray) -> np.ndarray: 2 return x</pre>		

Nama Fungsi	Tipe	Parameter
relu	Function	x: np.ndarray
Deskripsi: Menerapkan fungsi aktivasi ReLU $f(x) = \max(0, x)$. Nilai negatif menjadi 0, sedangkan nilai positif dipertahankan.		



```

1 def relu(x: np.ndarray) -> np.ndarray:
2     return np.maximum(x, 0)

```

Nama Fungsi	Tipe	Parameter
sigmoid	Function	x: np.ndarray
Deskripsi: Menerapkan fungsi sigmoid $f(x) = 1 / (1 + \exp(-x))$. Fungsi ini banyak digunakan pada gate LSTM.		



```

1 def sigmoid(x: np.ndarray) -> np.ndarray:
2     return 1.0 / (1.0 + np.exp(-x))

```

Nama Fungsi	Tipe	Parameter
tanh	Function	x: np.ndarray
Deskripsi: Menerapkan fungsi aktivasi hyperbolic tangent. Output berada pada rentang $[-1, 1]$.		

```

1 def tanh(x: np.ndarray) -> np.ndarray:
2     return np.tanh(x)

```

Nama Fungsi	Tipe	Parameter
softmax	Function	x: np.ndarray, axis: int = -1

Deskripsi:

Mengubah logits menjadi distribusi probabilitas. Implementasi menggunakan shifting dengan max(x) agar lebih stabil secara numerik.

```

1 def softmax(x: np.ndarray, axis: int = -1) -> np.ndarray:
2     shifted = x - np.max(x, axis=axis, keepdims=True)
3     exp = np.exp(shifted)
4     return exp / np.sum(exp, axis=axis, keepdims=True)

```

Nama Fungsi	Tipe	Parameter
activation_backward	Function	name: str None, pre_activation: np.ndarray, grad_output: np.ndarray

Deskripsi:

Menghitung gradien lokal fungsi aktivasi untuk proses backward propagation. Fungsi ini mendukung linear, relu, sigmoid, tanh, dan softmax. Untuk softmax, digunakan Jacobian-vector product agar gradien tetap efisien tanpa membentuk matriks Jacobian penuh.

```

1 def activation_backward(name: str | None, pre_activation: np.ndarray, grad_output: np.ndarray) -> np.ndarray:
2     if name is None or name == "linear":
3         return grad_output
4     if name == "relu":
5         return grad_output * (pre_activation > 0)
6     if name == "sigmoid":
7         activated = sigmoid(pre_activation)
8         return grad_output * activated * (1.0 - activated)
9     if name == "tanh":
10        activated = tanh(pre_activation)
11        return grad_output * (1.0 - activated * activated)
12    if name == "softmax":
13        probabilities = softmax(pre_activation, axis=-1)
14        dot = np.sum(grad_output * probabilities, axis=-1, keepdims=True)
15        return probabilities * (grad_output - dot)
16    raise ValueError(f"Unsupported activation: {name}")

```

Nama Fungsi	Tipe	Parameter
get_activation	Function	name: str None

Deskripsi:

Mengambil fungsi aktivasi berdasarkan nama. Mendukung linear, relu, sigmoid, tanh, dan softmax.

```

1 def get_activation(name: str | None):
2     if name is None or name == "linear":
3         return linear
4     if name == "relu":
5         return relu
6     if name == "sigmoid":
7         return sigmoid
8     if name == "tanh":
9         return tanh
10    if name == "softmax":
11        return softmax
12    raise ValueError(f"Unsupported activation: {name}")

```

2. CNN Scratch Layers

a. Class Conv2D (src/tubes2_ml/scratch/layers/[conv.py](#))

Atribut

Nama_Atribut	Tipe	Deskripsi
kernel	np.ndarray None	Bobot filter konvolusi dengan shape [kH, kW, C_in, C_out]
bias	np.ndarray None	Bias untuk setiap filter output.
strides	tuple[int, int]	Langkah perpindahan filter pada arah tinggi dan lebar.
padding	str	Jenis padding, yaitu valid atau same.
activation_name	str None	Nama fungsi aktivasi yang digunakan.
activation	Callable	Fungsi aktivasi hasil dari get_activation.
grad_kernel	np.ndarray	Gradien loss terhadap bobot kernel, dibuat saat backward dipanggil.
grad_bias	np.ndarray None	Gradien loss terhadap bias, dibuat saat backward dipanggil.
_cache	dict	Cache input, padding, dan pre-activation dari forward pass untuk kebutuhan backpropagation.

Method

Nama Fungsi	Tipe	Parameter
__init__	Method	kernel, bias, strides, padding, activation
load_keras_weights	Method	weights: list[np.ndarray] tuple[np.ndarray, ...]
forward	Method	x: np.ndarray
backward	Method	grad_output: np.ndarray
Deskripsi: Conv2D mengimplementasikan konvolusi 2D dengan parameter sharing. Input harus berbentuk (N, H, W, C). Pada setiap posisi spasial, layer mengambil patch input, menghitung dot product dengan kernel, menambahkan bias, lalu		

menerapkan aktivasi.

```
1 def __init__(self, kernel: np.ndarray | None = None, bias: np.ndarray | None = None, strides: int | tuple[int, int] = 1, padding: str = "valid", activation: str | None = None):
2     self.kernel = None if kernel is None else np.asarray(kernel, dtype=np.float32)
3     self.bias = None if bias is None else np.asarray(bias, dtype=np.float32)
4     self.strides = _to_pair(strides)
5     self.padding = padding.lower()
6     self.activation_name = activation
7     self.activation = get_activation(activation)
```

```
1 def load_keras_weights(self, weights: list[np.ndarray] | tuple[np.ndarray, ...]) -> None:
2     if len(weights) == 1:
3         self.kernel = np.asarray(weights[0], dtype=np.float32)
4         self.bias = None
5     elif len(weights) == 2:
6         self.kernel = np.asarray(weights[0], dtype=np.float32)
7         self.bias = np.asarray(weights[1], dtype=np.float32)
8     else:
9         raise ValueError("Conv2D weights must be [kernel] or [kernel, bias]")
```

```
1 def forward(self, x: np.ndarray) -> np.ndarray:
2     if self.kernel is None:
3         raise ValueError("Conv2D kernel is not loaded")
4
5     x = np.asarray(x, dtype=np.float32)
6     if x.ndim != 4:
7         raise ValueError("Conv2D input must have shape (N, H, W, C)")
8
9     batch_size, input_height, input_width, input_channels = x.shape
10    kernel_height, kernel_width, kernel_channels, output_channels = self.kernel.shape
11    if input_channels != kernel_channels:
12        raise ValueError("Input channels do not match Conv2D kernel channels")
13
14    stride_h, stride_w = self.strides
15    pad_top, pad_bottom, output_height = _compute_padding(
16        input_height, kernel_height, stride_h, self.padding
17    )
18    pad_left, pad_right, output_width = _compute_padding(
19        input_width, kernel_width, stride_w, self.padding
20    )
21
22    padded = np.pad(x, ((0, 0), (pad_top, pad_bottom), (pad_left, pad_right), (0, 0)), mode="constant")
23    output = np.zeros((batch_size, output_height, output_width, output_channels), dtype=np.float32)
24
25    for row in range(output_height):
26        row_start = row * stride_h
27        row_end = row_start + kernel_height
28        for col in range(output_width):
29            col_start = col * stride_w
30            col_end = col_start + kernel_width
31            patch = padded[:, row_start:row_end, col_start:col_end, :]
32            output[:, row, col, :] = np.tensordot(
33                patch,
34                self.kernel,
35                axes=((1, 2, 3), (0, 1, 2)),
36            )
37
38    if self.bias is not None:
39        output += self.bias.reshape(1, 1, 1, output_channels)
40
41    return self.activation(output)
```

```

1 def backward(self, grad_output: np.ndarray) -> np.ndarray:
2     if self.kernel is None:
3         raise ValueError("Conv2D kernel is not loaded")
4     if not hasattr(self, "_cache"):
5         raise ValueError("Conv2D backward called before forward")
6
7     grad = activation_backward(
8         self.activation_name,
9         self._cache["pre_activation"],
10        np.asarray(grad_output, dtype=np.float32),
11    )
12    padded = self._cache["padded"]
13    pad_top, pad_bottom, pad_left, pad_right = self._cache["padding"]
14    kernel_height, kernel_width, _, _ = self.kernel.shape
15    _, output_height, output_width, _ = grad.shape
16    stride_h, stride_w = self.strides
17
18    self.grad_kernel = np.zeros_like(self.kernel)
19    self.grad_bias = np.sum(grad, axis=(0, 1, 2)) if self.bias is not None else None
20    grad_padded = np.zeros_like(padded)
21
22    for row in range(output_height):
23        row_start = row * stride_h
24        row_end = row_start + kernel_height
25        for col in range(output_width):
26            col_start = col * stride_w
27            col_end = col_start + kernel_width
28            patch = padded[:, row_start:row_end, col_start:col_end, :]
29            grad_position = grad[:, row, col, :]
30            self.grad_kernel += np.tensordot(patch, grad_position, axes=([0], [0]))
31            grad_padded[:, row_start:row_end, col_start:col_end, :] += np.tensordot(
32                grad_position,
33                self.kernel,
34                axes=([1], [3]),
35            )
36
37    row_slice = slice(pad_top, grad_padded.shape[1] - pad_bottom if pad_bottom else None)
38    col_slice = slice(pad_left, grad_padded.shape[2] - pad_right if pad_right else None)
39    return grad_padded[:, row_slice, col_slice, :].reshape(self._cache["input_shape"])

```

b. Class LocallyConnected2D (src/tubes2_ml/scratch/layers/locally_connected.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
kernel	np.ndarray None	Bobot non-shared dengan shape (out_positions, patch_size, filters).
bias	np.ndarray None	Bias per posisi output atau per filter.
kernel_size	tuple[int, int] None	Ukuran kernel lokal
filters	int None	Banyak output channel.
strides	tuple[int, int]	Langkah sliding window.

padding	str	Jenis padding.
activation_name	str None	Nama aktivasi
activation	Callable	Fungsi aktivasi
grad_kernel	np.ndarray	Gradien loss terhadap bobot lokal setiap posisi output.
grad_bias	np.ndarray None	Gradien loss terhadap bias lokal atau bias per filter.
_cache	dict	Cache input, padding, dan pre-activation untuk backward pass.

Method

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Method	kernel, bias, strides, padding, activation
<code>load_keras_weights</code>	Method	weights: list[np.ndarray] tuple[np.ndarray, ...]
<code>forward</code>	Method	x: np.ndarray
<code>_reshape_bias</code>	Static Method	bias, output_height, output_width, output_channels
<code>backward</code>	Method	grad_output: np.ndarray
<code>_bias_backward</code>	Method	grad: np.ndarray

Deskripsi:

LocallyConnected2D mengimplementasikan konvolusi tanpa parameter sharing. Setiap posisi output memiliki bobot sendiri, sehingga jumlah parameter jauh lebih besar dibanding Conv2D.

```

1 def __init__(self, kernel: np.ndarray | None = None, bias: np.ndarray | None = None, kernel_size: int | tuple[int, int] | None = None, filters: int | None = None, strides: int | tuple[int, int] = 1, padding: str = "valid", activation: str | None = None):
2     self.kernel = None if kernel is None else np.asarray(kernel, dtype=np.float32)
3     self.bias = None if bias is None else np.asarray(bias, dtype=np.float32)
4     self.kernel_size = None if kernel_size is None else _to_pair(kernel_size)
5     self.filters = filters
6     self.strides = _to_pair(strides)
7     self.padding = padding.lower()
8     self.activation_name = activation
9     self.activation = get_activation(activation)
10

```



```

1 def load_keras_weights(self, weights: list[np.ndarray] | tuple[np.ndarray, ...]) -> None:
2
3     if len(weights) == 1:
4         self.kernel = np.asarray(weights[0], dtype=np.float32)
5         self.bias = None
6
7     elif len(weights) == 2:
8         self.kernel = np.asarray(weights[0], dtype=np.float32)
9         self.bias = np.asarray(weights[1], dtype=np.float32)
10
11     else:
12         raise ValueError("LocallyConnected2D weights must be [kernel] or [kernel, bias]")
13
14     if self.kernel.ndim != 3:
15         raise ValueError("LocallyConnected2D kernel must have shape (out_positions, patch_size, filters)")
16
17     self.filters = int(self.kernel.shape[-1])

```

```

1 def forward(self, x: np.ndarray) -> np.ndarray:
2
3     if self.kernel is None:
4         raise ValueError("LocallyConnected2D kernel is not loaded")
5     if self.kernel_size is None:
6         raise ValueError("kernel_size is required for LocallyConnected2D forward pass")
7
8     x = np.asarray(x, dtype=np.float32)
9     if x.ndim != 4:
10         raise ValueError("LocallyConnected2D input must have shape (N, H, W, C)")
11
12     batch_size, input_height, input_width, input_channels = x.shape
13     kernel_height, kernel_width = self.kernel_size
14     stride_h, stride_w = self.strides
15     pad_top, pad_bottom, output_height = _compute_padding(
16         input_height, kernel_height, stride_h, self.padding
17     )
18     pad_left, pad_right, output_width = _compute_padding(
19         input_width, kernel_width, stride_w, self.padding
20     )
21
22     expected_positions = output_height * output_width
23     patch_size = kernel_height * kernel_width * input_channels
24     if self.kernel.shape[0] != expected_positions:
25         raise ValueError("Kernel output positions do not match computed output size")
26     if self.kernel.shape[1] != patch_size:
27         raise ValueError("Kernel patch size does not match input channels and kernel_size")
28
29     output_channels = self.kernel.shape[2]
30     padded = np.pad(
31         x,
32         ((0, 0), (pad_top, pad_bottom), (pad_left, pad_right), (0, 0)),
33         mode="constant",
34     )
35     output = np.zeros((batch_size, output_height, output_width, output_channels), dtype=np.float32)
36
37     position = 0
38     for row in range(output_height):
39         row_start = row * stride_h
40         row_end = row_start + kernel_height
41         for col in range(output_width):
42             col_start = col * stride_w
43             col_end = col_start + kernel_width
44             patch = padded[:, row_start:row_end, col_start:col_end, :].reshape(batch_size, -1)
45             output[:, row, col, :] = patch @ self.kernel[position]
46             position += 1
47
48     if self.bias is not None:
49         output += self._reshape_bias(self.bias, output_height, output_width, output_channels)
50
51     return self.activation(output)

```

```

1 @staticmethod
2 def _reshape_bias(bias: np.ndarray, output_height: int, output_width: int, output_channels: int) -> np.ndarray:
3
4     if bias.shape == (output_height, output_width, output_channels):
5         return bias.reshape(1, output_height, output_width, output_channels)
6     if bias.shape == (output_height * output_width, output_channels):
7         return bias.reshape(1, output_height, output_width, output_channels)
8     if bias.shape == (output_channels,):
9         return bias.reshape(1, 1, 1, output_channels)
10    raise ValueError("Unsupported LocallyConnected2D bias shape")

```

```

1 def backward(self, grad_output: np.ndarray) -> np.ndarray:
2
3     if self.kernel is None:
4         raise ValueError("LocallyConnected2D kernel is not loaded")
5     if self.kernel_size is None:
6         raise ValueError("kernel_size is required for LocallyConnected2D backward pass")
7     if not hasattr(self, "_cache"):
8         raise ValueError("LocallyConnected2D backward called before forward")
9
10    grad = activation_backward(
11        self.activation_name,
12        self._cache["pre_activation"],
13        np.asarray(grad_output, dtype=np.float32),
14    )
15    padded = self._cache["padded"]
16    pad_top, pad_bottom, pad_left, pad_right = self._cache["padding"]
17    kernel_height, kernel_width = self.kernel_size
18    stride_h, stride_w = self.strides
19    _, output_height, output_width, output_channels = grad.shape
20
21    self.grad_kernel = np.zeros_like(self.kernel)
22    self.grad_bias = self._bias_backward(grad) if self.bias is not None else None
23    grad_padded = np.zeros_like(padded)
24
25    position = 0
26    for row in range(output_height):
27        row_start = row * stride_h
28        row_end = row_start + kernel_height
29        for col in range(output_width):
30            col_start = col * stride_w
31            col_end = col_start + kernel_width
32            patch = padded[:, row_start:row_end, col_start:col_end, :].reshape(padded.shape[0], -1)
33            grad_position = grad[:, row, col, :]
34            self.grad_kernel[position] = patch.T @ grad_position
35            grad_patch = grad_position @ self.kernel[position].T
36            grad_padded[:, row_start:row_end, col_start:col_end, :] += grad_patch.reshape(
37                padded.shape[0],
38                kernel_height,
39                kernel_width,
40                -1,
41            )
42            position += 1
43
44    row_slice = slice(pad_top, grad_padded.shape[1] - pad_bottom if pad_bottom else None)
45    col_slice = slice(pad_left, grad_padded.shape[2] - pad_right if pad_right else None)
46    return grad_padded[:, row_slice, col_slice, :].reshape(self._cache["input_shape"])

```

```

1 def _bias_backward(self, grad: np.ndarray) -> np.ndarray:
2     _, output_height, output_width, output_channels = grad.shape
3     if self.bias.shape == (output_height, output_width, output_channels):
4         return np.sum(grad, axis=0)
5     if self.bias.shape == (output_height * output_width, output_channels):
6         return np.sum(grad, axis=0).reshape(output_height * output_width, output_channels)
7     if self.bias.shape == (output_channels,):
8         return np.sum(grad, axis=(0, 1, 2))
9     raise ValueError("Unsupported LocallyConnected2D bias shape")

```

c. Class Pooling2D (src/tubes2_ml/scratch/layers/pooling.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
pool_size	tuple[int, int]	Ukuran window pooling.
bias	tuple[int, int]	Langkah perpindahan window pooling
padding	str	Jenis padding.
mode	str	Jenis pooling, yaitu max atau average.
_cache	dict	Cache input, padding, dan output pooling untuk menghitung gradien pooling.

Method

Nama Fungsi	Tipe	Parameter
__init__	Method	pool_size, strides, padding, mode
forward	Method	x: np.ndarray
backward	Method	grad_output: np.ndarray
Deskripsi: Pooling2D melakukan reduksi spasial pada tiap channel. Jika mode="max", output berisi nilai maksimum pada window. Jika mode="average", output berisi rata-rata window.		

```

1 def __init__(self, pool_size: int | tuple[int, int] = (2, 2), strides: int | tuple[int, int] | None = None, padding: str = "valid", mode: str = "max"):
2
3     if mode not in {"max", "average"}:
4         raise ValueError("mode must be either 'max' or 'average'")
5
6     self.pool_size = _to_pair(pool_size)
7     self.strides = _to_pair(strides if strides is not None else pool_size)
8     self.padding = padding.lower()
9     self.mode = mode

```

```

1 def forward(self, x: np.ndarray) -> np.ndarray:
2     x = np.asarray(x, dtype=np.float32)
3     if x.ndim != 4:
4         raise ValueError("Pooling2D input must have shape (N, H, W, C)")
5
6     batch_size, input_height, input_width, channels = x.shape
7     pool_h, pool_w = self.pool_size
8     stride_h, stride_w = self.strides
9     pad_top, pad_bottom, output_height = _compute_padding(
10         input_height, pool_h, stride_h, self.padding
11     )
12     pad_left, pad_right, output_width = _compute_padding(
13         input_width, pool_w, stride_w, self.padding
14     )
15
16     pad_value = -np.inf if self.mode == "max" else 0.0
17     padded = np.pad(x, ((0, 0), (pad_top, pad_bottom), (pad_left, pad_right), (0, 0)), mode="constant", constant_values=pad_value)
18     output = np.zeros((batch_size, output_height, output_width, channels), dtype=np.float32)
19
20     for row in range(output_height):
21         row_start = row * stride_h
22         row_end = row_start + pool_h
23         for col in range(output_width):
24             col_start = col * stride_w
25             col_end = col_start + pool_w
26             patch = padded[:, row_start:row_end, col_start:col_end, :]
27             if self.mode == "max":
28                 output[:, row, col, :] = np.max(patch, axis=(1, 2))
29             else:
30                 output[:, row, col, :] = np.mean(patch, axis=(1, 2))
31     return output

```

```

1 def backward(self, grad_output: np.ndarray) -> np.ndarray:
2     if not hasattr(self, "_cache"):
3         raise ValueError("Pooling2D backward called before forward")
4
5     grad = np.asarray(grad_output, dtype=np.float32)
6     padded = self._cache["padded"]
7     pad_top, pad_bottom, pad_left, pad_right = self._cache["padding"]
8     pool_h, pool_w = self.pool_size
9     stride_h, stride_w = self.strides
10    _, output_height, output_width, _ = grad.shape
11    grad_padded = np.zeros_like(padded)
12
13    for row in range(output_height):
14        row_start = row * stride_h
15        row_end = row_start + pool_h
16        for col in range(output_width):
17            col_start = col * stride_w
18            col_end = col_start + pool_w
19            patch = padded[:, row_start:row_end, col_start:col_end, :]
20            grad_position = grad[:, row, col, :][:, np.newaxis, np.newaxis, :]
21            if self.mode == "max":
22                maxima = np.max(patch, axis=(1, 2), keepdims=True)
23                mask = patch == maxima
24                counts = np.sum(mask, axis=(1, 2), keepdims=True)
25                grad_padded[:, row_start:row_end, col_start:col_end, :] += mask * grad_position / counts
26            else:
27                grad_padded[:, row_start:row_end, col_start:col_end, :] += grad_position / (pool_h * pool_w)
28
29    row_slice = slice(pad_top, grad_padded.shape[1] - pad_bottom if pad_bottom else None)
30    col_slice = slice(pad_left, grad_padded.shape[2] - pad_right if pad_right else None)
31    return grad_padded[:, row_slice, col_slice, :].reshape(self._cache["input_shape"])

```

d. Class MaxPooling2D (src/tubes2_ml/scratch/layers/pooling.py)

Method

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Method	<code>pool_size</code> , <code>strides</code> , <code>padding</code> , <code>mode</code>
Deskripsi: Turunan dari Pooling2D dengan <code>mode="max"</code> . Digunakan untuk mengambil aktivasi terbesar pada tiap window.		

e. Class `AveragePooling2D` (`src/tubes2_ml/scratch/layers/pooling.py`)

Method

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Method	<code>pool_size</code> , <code>strides</code> , <code>padding</code>
Deskripsi: Turunan dari Pooling2D dengan <code>mode="average"</code> . Digunakan untuk mengambil nilai rata-rata pada tiap window.		

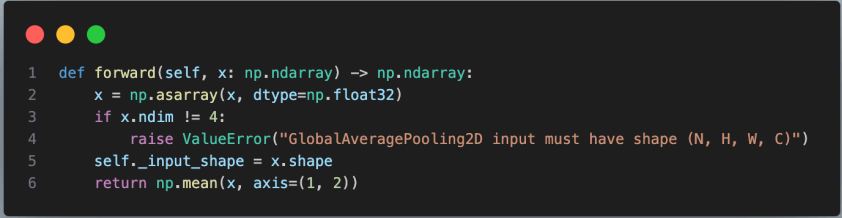
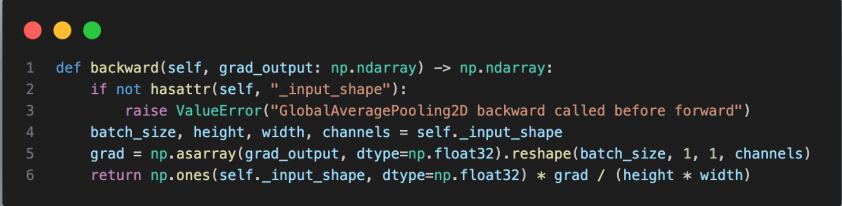
f. Class `GlobalAveragePooling2D` (`src/tubes2_ml/scratch/layers/pooling.py`)

Atribut

Nama_Atribut	Tipe	Deskripsi
<code>_input_shape</code>	<code>tuple[int, ...]</code>	Shape input terakhir, dipakai untuk menyebarkan gradien ke dimensi spasial.

Method

Nama Fungsi	Tipe	Parameter
<code>forward</code>	Method	<code>x</code> : <code>np.ndarray</code>

backward	Method	grad_output: np.ndarray
Deskripsi: Mereduksi seluruh dimensi spasial (H, W) menjadi satu nilai rata-rata per channel. Output berbentuk (N, C).		
 <pre> 1 def forward(self, x: np.ndarray) -> np.ndarray: 2 x = np.asarray(x, dtype=np.float32) 3 if x.ndim != 4: 4 raise ValueError("GlobalAveragePooling2D input must have shape (N, H, W, C)") 5 self._input_shape = x.shape 6 return np.mean(x, axis=(1, 2)) </pre>		
 <pre> 1 def backward(self, grad_output: np.ndarray) -> np.ndarray: 2 if not hasattr(self, "_input_shape"): 3 raise ValueError("GlobalAveragePooling2D backward called before forward") 4 batch_size, height, width, channels = self._input_shape 5 grad = np.asarray(grad_output, dtype=np.float32).reshape(batch_size, 1, 1, channels) 6 return np.ones(self._input_shape, dtype=np.float32) * grad / (height * width) </pre>		

g. Class GlobalMaxPooling2D (src/tubes2_ml/scratch/layers/[pooling.py](#))

Atribut

Nama_Atribut	Tipe	Deskripsi
_input	np.ndarray	Input terakhir dari forward pass untuk menentukan posisi maksimum saat backward.

Method

Nama Fungsi	Tipe	Parameter
forward	Method	x: np.ndarray
backward	Method	grad_output: np.ndarray
Deskripsi: Mereduksi seluruh dimensi spasial (H, W) menjadi nilai maksimum per channel. Output berbentuk (N, C).		

```

1 def forward(self, x: np.ndarray) -> np.ndarray:
2     x = np.asarray(x, dtype=np.float32)
3     if x.ndim != 4:
4         raise ValueError("GlobalMaxPooling2D input must have shape (N, H, W, C)")
5     self._input = x
6     output = np.max(x, axis=(1, 2))
7     self._output = output
8     return output

1 def backward(self, grad_output: np.ndarray) -> np.ndarray:
2     if not hasattr(self, "_input") or not hasattr(self, "_output"):
3         raise ValueError("GlobalMaxPooling2D backward called before forward")
4     grad = np.asarray(grad_output, dtype=np.float32)[: , np.newaxis, np.newaxis, :]
5     maxima = self._output[:, np.newaxis, np.newaxis, :]
6     mask = self._input == maxima
7     counts = np.sum(mask, axis=(1, 2), keepdims=True)
8     return mask * grad / counts

```

h. Class Flatten (src/tubes2_ml/scratch/layers/[flatten.py](#))

Atribut

Nama_Atribut	Tipe	Deskripsi
_input_shape	tuple[int, ...]	Shape input terakhir agar gradien dapat dikembalikan ke bentuk semula..

Method

Nama Fungsi	Tipe	Parameter
forward	Method	x: np.ndarray
backward	Method	grad_output: np.ndarray
Deskripsi: Mengubah tensor dengan batch dimension menjadi matriks 2D (N, -1) menggunakan urutan row-major / C order, konsisten dengan Keras.		



3. Dense dan Embedding Scratch Layers

a. Class Dense (src/tubes2_ml/scratch/layers/dense.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
weights	np.ndarray None	Matriks bobot Dense.
bias	np.ndarray None	Bias Dense.
activation_name	str None	Nama aktivasi.
activation	Callable	Fungsi aktivasi.
grad_weights	np.ndarray	Gradien loss terhadap matriks bobot, dibuat saat backward dipanggil.
grad_bias	np.ndarray None	Gradien loss terhadap bias Dense, dibuat saat backward dipanggil.
_input	np.ndarray	Input terakhir dari forward pass.
_pre_activation	np.ndarray	Output linear sebelum aktivasi, dipakai untuk menghitung turunan aktivasi.

Method

Nama Fungsi	Tipe	Parameter
-------------	------	-----------

<code>__init__</code>	Method	weights, bias, activation
<code>load_keras_weights</code>	Method	weights: list[np.ndarray] tuple[np.ndarray, ...]
<code>forward</code>	Method	x: np.ndarray
<code>backward</code>	Method	grad_output: np.ndarray

Deskripsi:

Dense menghitung operasi linear output = $x @ W + b$, lalu menerapkan aktivasi. Layer ini digunakan pada CNN classifier, feature projection captioning, dan output vocabulary.

```

1 def __init__(self, weights: np.ndarray | None = None, bias: np.ndarray | None = None, activation: str | None = None):
2     self.weights = None if weights is None else np.asarray(weights, dtype=np.float32)
3     self.bias = None if bias is None else np.asarray(bias, dtype=np.float32)
4     self.activation_name = activation
5     self.activation = get_activation(activation)

```

```

1 def load_keras_weights(self, weights: list[np.ndarray] | tuple[np.ndarray, ...]) -> None:
2     if len(weights) == 1:
3         self.weights = np.asarray(weights[0], dtype=np.float32)
4         self.bias = None
5     elif len(weights) == 2:
6         self.weights = np.asarray(weights[0], dtype=np.float32)
7         self.bias = np.asarray(weights[1], dtype=np.float32)
8     else:
9         raise ValueError("Dense weights must be [kernel] or [kernel, bias]")

```

```

1 def forward(self, x: np.ndarray) -> np.ndarray:
2     if self.weights is None:
3         raise ValueError("Dense weights are not loaded")
4
5     x = np.asarray(x, dtype=np.float32)
6     output = x @ self.weights
7     if self.bias is not None:
8         output += self.bias
9
10    return self.activation(output)

```

```

1 def backward(self, grad_output: np.ndarray) -> np.ndarray:
2     if self.weights is None:
3         raise ValueError("Dense weights are not loaded")
4     if not hasattr(self, "_input") or not hasattr(self, "_pre_activation"):
5         raise ValueError("Dense backward called before forward")
6
7     grad = activation_backward(
8         self.activation_name,
9         self._pre_activation,
10        np.asarray(grad_output, dtype=np.float32),
11    )
12    input_shape = self._input.shape
13    flat_input = self._input.reshape(-1, self.weights.shape[0])
14    flat_grad = grad.reshape(-1, self.weights.shape[1])
15
16    self.grad_weights = flat_input.T @ flat_grad
17    self.grad_bias = np.sum(flat_grad, axis=0) if self.bias is not None else None
18    grad_input = flat_grad @ self.weights.T
19    return grad_input.reshape(input_shape)

```

b. Class Embedding (src/tubes2_ml/scratch/layers/[embedding.py](#))

Atribut

Nama_Atribut	Tipe	Deskripsi
weights	np.ndarray None	Matriks embedding dengan shape (vocab_size, embed_dim)
grad_weights	np.ndarray	Gradien loss terhadap matriks embedding, dibuat saat backward dipanggil.
_ids	np.ndarray	Token id dari forward pass terakhir untuk akumulasi gradien embedding.

Method

Nama Fungsi	Tipe	Parameter
__init__	Method	weights
load_keras_weights	Method	weights: list[np.ndarray] tuple[np.ndarray, ...]
forward	Method	token_ids: np.ndarray
backward	Method	grad_output: np.ndarray

Deskripsi:

Embedding mengubah token id menjadi vektor embedding. Input adalah token integer, output adalah tensor embedding sesuai shape input token ditambah dimensi embedding.

```
1 def __init__(self, weights: np.ndarray | None = None):
2     self.weights = None if weights is None else np.asarray(weights, dtype=np.float32)
```

```
1 def load_keras_weights(self, weights: list[np.ndarray] | tuple[np.ndarray, ...]) -> None:
2     if len(weights) != 1:
3         raise ValueError("Embedding weights must be [embeddings]")
4     self.weights = np.asarray(weights[0], dtype=np.float32)
```

```
1 def forward(self, token_ids: np.ndarray) -> np.ndarray:
2     if self.weights is None:
3         raise ValueError("Embedding weights are not loaded")
4
5     ids = np.asarray(token_ids, dtype=np.int64)
6     if np.any(ids < 0) or np.any(ids >= self.weights.shape[0]):
7         raise ValueError("Token id out of embedding vocabulary range")
8     return self.weights[ids]
```

```
1 def backward(self, grad_output: np.ndarray) -> None:
2     if self.weights is None:
3         raise ValueError("Embedding weights are not loaded")
4     if not hasattr(self, "_ids"):
5         raise ValueError("Embedding backward called before forward")
6
7     self.grad_weights = np.zeros_like(self.weights)
8     np.add.at(self.grad_weights, self._ids, np.asarray(grad_output, dtype=np.float32))
9     return None
```

4. RNN dan LSTM Scratch Layers

a. Class SimpleRNN (src/tubes2_ml/scratch/layers/recurrent.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
kernel	np.ndarray None	Bobot input-to-hidden dari Keras.
recurrent_kernel	np.ndarray None	Bobot hidden-to-hidden dari Keras.
bias	np.ndarray None	Bias SimpleRNN.
units	int	Property yang menunjukkan jumlah hidden unit.
grad_kernel	np.ndarray	Gradien loss terhadap bobot input-to-hidden.
grad_recurrent_kernel	np.ndarray	Gradien loss terhadap bobot hidden-to-hidden.
grad_bias	np.ndarray None	Gradien loss terhadap bias SimpleRNN.
_cache	dict	Cache input, state sebelumnya, dan state setiap timestep untuk BPTT.

Method

Nama Fungsi	Tipe	Parameter
__init__	Method	kernel, recurrent_kernel, bias
load_keras_weights	Method	weights: list[np.ndarray] tuple[np.ndarray, ...]
units	Property	-
forward	Method	x, initial_state, return_state
backward	Method	grad_outputs, grad_final_state
Deskripsi: SimpleRNN melakukan forward propagation recurrent dengan rumus:		

```

1 def __init__(
2     self,
3     kernel: np.ndarray | None = None,
4     recurrent_kernel: np.ndarray | None = None,
5     bias: np.ndarray | None = None,
6 ):
7     self.kernel = None if kernel is None else np.asarray(kernel, dtype=np.float32)
8     self.recurrent_kernel = (
9         None if recurrent_kernel is None else np.asarray(recurrent_kernel, dtype=np.float32)
10    )
11    self.bias = None if bias is None else np.asarray(bias, dtype=np.float32)

```

```

1 def load_keras_weights(self, weights: list[np.ndarray] | tuple[np.ndarray, ...]) -> None:
2     if len(weights) != 3:
3         raise ValueError("SimpleRNN weights must be [kernel, recurrent_kernel, bias]")
4     self.kernel = np.asarray(weights[0], dtype=np.float32)
5     self.recurrent_kernel = np.asarray(weights[1], dtype=np.float32)
6     self.bias = np.asarray(weights[2], dtype=np.float32)

```

```

1 @property
2 def units(self) -> int:
3     if self.recurrent_kernel is None:
4         raise ValueError("SimpleRNN weights are not loaded")
5     return int(self.recurrent_kernel.shape[0])

```

```

1 def forward(
2     self,
3     x: np.ndarray,
4     initial_state: np.ndarray | None = None,
5     return_state: bool = False,
6 ):
7     if self.kernel is None or self.recurrent_kernel is None:
8         raise ValueError("SimpleRNN weights are not loaded")
9
10    inputs = np.asarray(x, dtype=np.float32)
11    if inputs.ndim != 3:
12        raise ValueError("SimpleRNN input must have shape (batch, timesteps, features)")
13
14    batch_size, timesteps, _ = inputs.shape
15    if initial_state is None:
16        h_t = np.zeros((batch_size, self.units), dtype=np.float32)
17    else:
18        h_t = np.asarray(initial_state, dtype=np.float32)
19
20    outputs: list[np.ndarray] = []
21    bias = 0.0 if self.bias is None else self.bias
22    for timestep in range(timesteps):
23        h_t = tanh(inputs[:, timestep, :] @ self.kernel + h_t @ self.recurrent_kernel + bias)
24        outputs.append(h_t)
25
26    sequence = np.stack(outputs, axis=1) if outputs else np.empty((batch_size, 0, self.units), dtype=np.float32)
27    if return_state:
28        return sequence, h_t
29    return sequence

```

```

1 def backward(
2     self,
3     grad_outputs: np.ndarray,
4     grad_final_state: np.ndarray | None = None,
5 ) -> tuple[np.ndarray, np.ndarray]:
6     if self.kernel is None or self.recurrent_kernel is None:
7         raise ValueError("SimpleRNN weights are not loaded")
8     if not hasattr(self, "_cache"):
9         raise ValueError("SimpleRNN backward called before forward")
10
11     inputs = self._cache["inputs"]
12     prev_states = self._cache["prev_states"]
13     states = self._cache["states"]
14     grad_sequence = np.asarray(grad_outputs, dtype=np.float32)
15     batch_size, timesteps, input_dim = inputs.shape
16
17     self.grad_kernel = np.zeros_like(self.kernel)
18     self.grad_recurrent_kernel = np.zeros_like(self.recurrent_kernel)
19     self.grad_bias = np.zeros_like(self.bias) if self.bias is not None else None
20     grad_inputs = np.zeros_like(inputs)
21     grad_h_next = (
22         np.zeros((batch_size, self.units), dtype=np.float32)
23         if grad_final_state is None
24         else np.asarray(grad_final_state, dtype=np.float32)
25     )
26
27     for timestep in range(timesteps - 1, -1, -1):
28         grad_h = grad_sequence[:, timestep, :] + grad_h_next
29         h_t = states[timestep]
30         prev_h = prev_states[timestep]
31         grad_z = grad_h * (1.0 - h_t * h_t)
32         self.grad_kernel += inputs[:, timestep, :].T @ grad_z
33         self.grad_recurrent_kernel += prev_h.T @ grad_z
34         if self.grad_bias is not None:
35             self.grad_bias += np.sum(grad_z, axis=0)
36         grad_inputs[:, timestep, :] = grad_z @ self.kernel.T
37         grad_h_next = grad_z @ self.recurrent_kernel.T
38
39     return grad_inputs.reshape(batch_size, timesteps, input_dim), grad_h_next

```

Input berbentuk (batch, timesteps, features). Jika initial_state tidak diberikan, hidden state awal diisi nol.

b. Class LSTM (src/tubes2_ml/scratch/layers/recurrent.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
kernel	np.ndarray None	Bobot input untuk semua gate LSTM.
recurrent_kernel	np.ndarray None	Bobot recurrent untuk semua gate LSTM.
bias	np.ndarray None	Bias LSTM.

units	int	Property jumlah hidden unit.
grad_kernel	np.ndarray	Gradien loss terhadap bobot input untuk semua gate LSTM.
grad_recurrent_kernel	np.ndarray	Gradien loss terhadap bobot recurrent untuk semua gate LSTM.
grad_bias	np.ndarray None	Gradien loss terhadap bias LSTM.
_cache	dict	Cache input dan gate tiap timestep untuk BPTT LSTM.

Method

Nama Fungsi	Tipe	Parameter
__init__	Method	kernel, recurrent_kernel, bias
load_keras_weights	Method	weights: list[np.ndarray] tuple[np.ndarray, ...]
units	Property	-
forward	Method	x, initial_state, return_state
backward	Method	grad_outputs, grad_final_state

Deskripsi:

LSTM melakukan forward propagation dengan empat gate sesuai format bobot Keras: input gate, forget gate, candidate cell, dan output gate.

```

1 def __init__(
2     self,
3     kernel: np.ndarray | None = None,
4     recurrent_kernel: np.ndarray | None = None,
5     bias: np.ndarray | None = None,
6 ):
7     self.kernel = None if kernel is None else np.asarray(kernel, dtype=np.float32)
8     self.recurrent_kernel = (
9         None if recurrent_kernel is None else np.asarray(recurrent_kernel, dtype=np.float32)
10    )
11    self.bias = None if bias is None else np.asarray(bias, dtype=np.float32)

```



```

1 def load_keras_weights(self, weights: list[np.ndarray] | tuple[np.ndarray, ...]) -> None:
2     if len(weights) != 3:
3         raise ValueError("LSTM weights must be [kernel, recurrent_kernel, bias]")
4     self.kernel = np.asarray(weights[0], dtype=np.float32)
5     self.recurrent_kernel = np.asarray(weights[1], dtype=np.float32)
6     self.bias = np.asarray(weights[2], dtype=np.float32)

```

```

1 @property
2 def units(self) -> int:
3     if self.recurrent_kernel is None:
4         raise ValueError("LSTM weights are not loaded")
5     return int(self.recurrent_kernel.shape[0])

```

```

1 def forward(
2     self,
3     x: np.ndarray,
4     initial_state: tuple[np.ndarray, np.ndarray] | None = None,
5     return_state: bool = False,
6 ):
7     if self.kernel is None or self.recurrent_kernel is None:
8         raise ValueError("LSTM weights are not loaded")
9
10    inputs = np.asarray(x, dtype=np.float32)
11    if inputs.ndim != 3:
12        raise ValueError("LSTM input must have shape (batch, timesteps, features)")
13
14    batch_size, timesteps, _ = inputs.shape
15    if initial_state is None:
16        h_t = np.zeros((batch_size, self.units), dtype=np.float32)
17        c_t = np.zeros((batch_size, self.units), dtype=np.float32)
18    else:
19        h_t = np.asarray(initial_state[0], dtype=np.float32)
20        c_t = np.asarray(initial_state[1], dtype=np.float32)
21
22    outputs: list[np.ndarray] = []
23    bias = 0.0 if self.bias is None else self.bias
24    for timestep in range(timesteps):
25        z = inputs[:, timestep, :] @ self.kernel + h_t @ self.recurrent_kernel + bias
26        z_i, z_f, z_c, z_o = np.split(z, 4, axis=-1)
27        i_t = sigmoid(z_i)
28        f_t = sigmoid(z_f)
29        c_hat_t = tanh(z_c)
30        o_t = sigmoid(z_o)
31        c_t = f_t * c_t + i_t * c_hat_t
32        h_t = o_t * tanh(c_t)
33        outputs.append(h_t)
34
35    sequence = np.stack(outputs, axis=1) if outputs else np.empty((batch_size, 0, self.units), dtype=np.float32)
36    if return_state:
37        return sequence, h_t, c_t
38    return sequence

```

```

1 def backward(
2     self,
3     grad_outputs: np.ndarray,
4     grad_final_state: tuple[np.ndarray, np.ndarray] | None = None,
5 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
6     if self.kernel is None or self.recurrent_kernel is None:
7         raise ValueError("LSTM weights are not loaded")
8     if not hasattr(self, "_cache"):
9         raise ValueError("LSTM backward called before forward")
10
11     inputs = self._cache["inputs"]
12     step_caches = self._cache["step_caches"]
13     grad_sequence = np.asarray(grad_outputs, dtype=np.float32)
14     batch_size, timesteps, input_dim = inputs.shape
15
16     self.grad_kernel = np.zeros_like(self.kernel)
17     self.grad_recurrent_kernel = np.zeros_like(self.recurrent_kernel)
18     self.grad_bias = np.zeros_like(self.bias) if self.bias is not None else None
19     grad_inputs = np.zeros_like(inputs)
20     if grad_final_state is None:
21         grad_h_next = np.zeros((batch_size, self.units), dtype=np.float32)
22         grad_c_next = np.zeros((batch_size, self.units), dtype=np.float32)
23     else:
24         grad_h_next = np.asarray(grad_final_state[0], dtype=np.float32)
25         grad_c_next = np.asarray(grad_final_state[1], dtype=np.float32)
26
27     for timestep in range(timesteps - 1, -1, -1):
28         cache = step_caches[timestep]
29         grad_h = grad_sequence[:, timestep, :] + grad_h_next
30         tanh_c = tanh(cache["c"])
31         grad_o = grad_h * tanh_c
32         grad_c = grad_h * cache["o"] * (1.0 - tanh_c * tanh_c) + grad_c_next
33         grad_f = grad_c * cache["prev_c"]
34         grad_prev_c = grad_c * cache["f"]
35         grad_i = grad_c * cache["c_hat"]
36         grad_c_hat = grad_c * cache["i"]
37
38         grad_z_i = grad_i * cache["i"] * (1.0 - cache["i"])
39         grad_z_f = grad_f * cache["f"] * (1.0 - cache["f"])
40         grad_z_c = grad_c_hat * (1.0 - cache["c_hat"]) * cache["c_hat"]
41         grad_z_o = grad_o * cache["o"] * (1.0 - cache["o"])
42         grad_z = np.concatenate([grad_z_i, grad_z_f, grad_z_c, grad_z_o], axis=-1)
43
44         self.grad_kernel += inputs[:, timestep, :].T @ grad_z
45         self.grad_recurrent_kernel += cache["prev_h"].T @ grad_z
46         if self.grad_bias is not None:
47             self.grad_bias += np.sum(grad_z, axis=0)
48         grad_inputs[:, timestep, :] = grad_z @ self.kernel.T
49         grad_h_next = grad_z @ self.recurrent_kernel.T
50         grad_c_next = grad_prev_c
51
52     return grad_inputs.reshape(batch_size, timesteps, input_dim), grad_h_next, grad_c_next

```

Jika initial_state tidak diberikan, h0 dan c0 diisi nol.

5. Scratch Model Wrappers

a. Class ScratchCNNClassifier (src/tubes2_ml/scratch/models/cnn_classifier.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
layers	list	Daftar layer scratch yang

		dijalankan berurutan.
--	--	-----------------------

Method

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Method	layers: list
<code>forward</code>	Method	x: np.ndarray
<code>predict</code>	Property	x: np.ndarray
<code>count_parameters</code>	Method	-

Deskripsi:

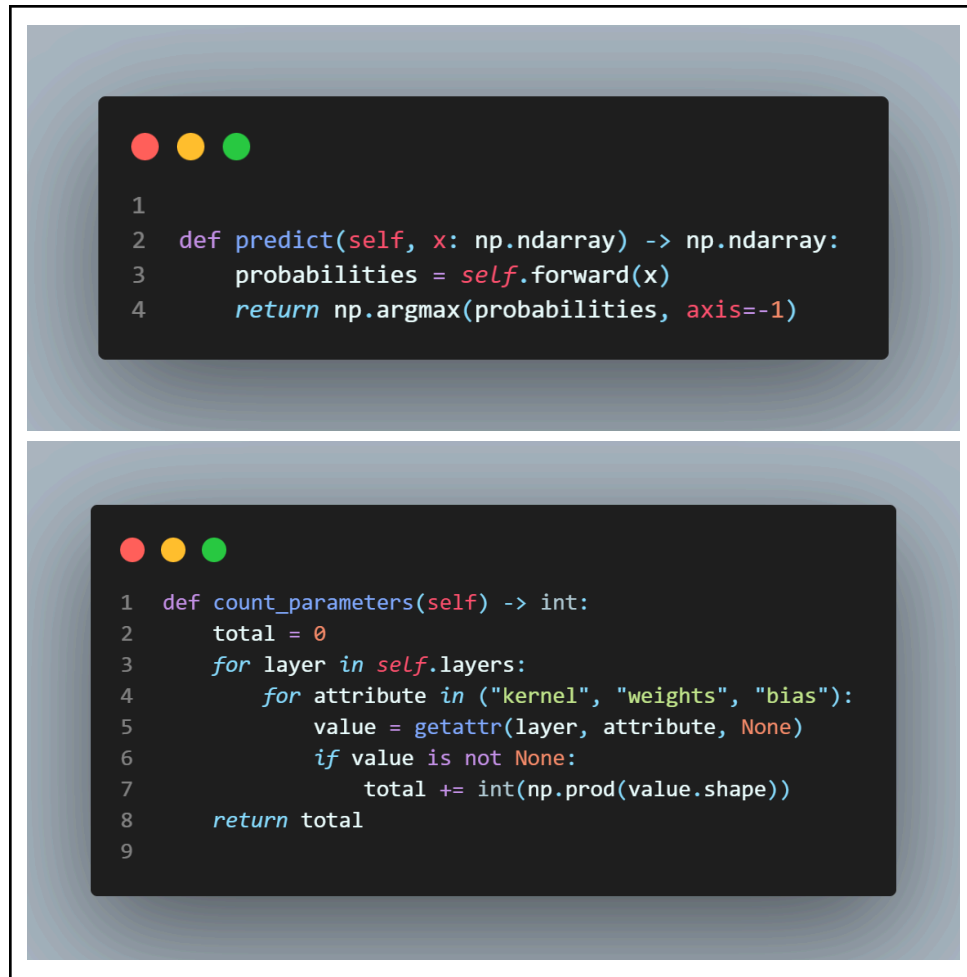
Wrapper untuk menjalankan model CNN dari scratch. Method `forward` melewati input ke semua layer, `predict` mengambil argmax probabilitas kelas, dan `count_parameters` menghitung total parameter.



```
1 def __init__(self, layers: list):
2     self.layers = layers
```



```
1 def forward(self, x: np.ndarray) -> np.ndarray:
2     output = np.asarray(x, dtype=np.float32)
3     for layer in self.layers:
4         output = layer.forward(output)
5     return output
```



b. Class ScratchRNNCaptioner (src/tubes2_ml/scratch/models/rnn_captioner.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
feature_projection	Dense	Dense layer untuk memproyeksikan CNN feature ke embedding dimension.
embedding	Embedding	Token embedding layer.
recurrent_layers	list[SimpleRNN]	Daftar layer SimpleRNN scratch.
output_dense	Dense	Dense output menuju vocab logits.

Method

Nama Fungsi	Tipe	Parameter
<code>__init__</code>	Method	<code>feature_projection</code> , <code>embedding</code> , <code>recurrent_layers</code> , <code>output_dense</code>
<code>from_keras_model</code>	Class Method	<code>keras_model</code>
<code>forward</code>	Method	<code>image_features</code> , <code>caption_input_ids</code>

Deskripsi:

ScratchRNNCaptioner membangun decoder captioning RNN dari bobot model Keras. Feature CNN diproyeksikan sebagai timestep awal, lalu digabung dengan embedding token. Output berupa probabilitas vocabulary.

```

1  def __init__(
2      self,
3      feature_projection: Dense,
4      embedding: Embedding,
5      recurrent_layers: list[SimpleRNN],
6      output_dense: Dense,
7  ):
8      if not recurrent_layers:
9          raise ValueError("At least one SimpleRNN layer is required")
10     self.feature_projection = feature_projection
11     self.embedding = embedding
12     self.recurrent_layers = recurrent_layers
13     self.output_dense = output_dense

```

```

1  @classmethod
2  def from_keras_model(cls, keras_model) -> "ScratchRNNCaptioner":
3      feature_projection = Dense()
4      feature_projection.load_keras_weights(keras_model.get_layer("feature_projection").get_weights())
5
6      embedding = Embedding()
7      embedding.load_keras_weights(keras_model.get_layer("token_embedding").get_weights())
8
9      recurrent_layers: list[SimpleRNN] = []
10     index = 1
11     while True:
12         try:
13             keras_layer = keras_model.get_layer(f"rnn_{index}")
14         except ValueError:
15             break
16         layer = SimpleRNN()
17         layer.load_keras_weights(keras_layer.get_weights())
18         recurrent_layers.append(layer)
19         index += 1
20
21     output_dense = Dense()
22     output_dense.load_keras_weights(keras_model.get_layer("word_logits").get_weights())
23     return cls(feature_projection, embedding, recurrent_layers, output_dense)

```

```

1 def forward(self, image_features: np.ndarray, caption_input_ids: np.ndarray) -> np.ndarray:
2     features = np.asarray(image_features, dtype=np.float32)
3     if features.ndim == 1:
4         features = features[np.newaxis, :]
5
6     token_ids = np.asarray(caption_input_ids, dtype=np.int32)
7     if token_ids.ndim == 1:
8         token_ids = token_ids[np.newaxis, :]
9
10    feature_timestep = self.feature_projection.forward(features)[: , np.newaxis, :]
11    token_embeddings = self.embedding.forward(token_ids)
12    sequence = np.concatenate([feature_timestep, token_embeddings], axis=1)
13
14    x = sequence
15    for layer in self.recurrent_layers:
16        x = layer.forward(x)
17
18    token_outputs = x[:, 1:, :]
19    logits = self.output_dense.forward(token_outputs)
20    return softmax(logits, axis=-1)

```

c. Class ScratchLSTMCaptioner (src/tubes2_ml/scratch/models/lstm_captioner.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
feature_projection	Dense	Dense layer untuk memproyeksikan CNN feature ke embedding dimension.
embedding	Embedding	Token embedding layer.
recurrent_layers	list[LSTM]	Daftar layer LSTM scratch.
output_dense	Dense	Dense output menuju vocab logits.

Method

Nama Fungsi	Tipe	Parameter
__init__	Method	feature_projection, embedding, recurrent_layers, output_dense
from_keras_model	Class Method	keras_model
forward	Method	image_features, caption_input_ids
Deskripsi: ScratchLSTMCaptioner membangun decoder captioning LSTM dari bobot		

model Keras. Alurnya sama dengan ScratchRNNCaptioner, tetapi recurrent cell yang digunakan adalah LSTM.

```
1 def __init__(
2     self,
3     feature_projection: Dense,
4     embedding: Embedding,
5     recurrent_layers: list[LSTM],
6     output_dense: Dense,
7 ):
8     if not recurrent_layers:
9         raise ValueError("At least one LSTM layer is required")
10    self.feature_projection = feature_projection
11    self.embedding = embedding
12    self.recurrent_layers = recurrent_layers
13    self.output_dense = output_dense
14
```

```
1 @classmethod
2 def from_keras_model(cls, keras_model) -> "ScratchLSTMCaptioner":
3     feature_projection = Dense()
4     feature_projection.load_keras_weights(keras_model.get_layer("feature_projection").get_weights())
5
6     embedding = Embedding()
7     embedding.load_keras_weights(keras_model.get_layer("token_embedding").get_weights())
8
9     recurrent_layers: list[LSTM] = []
10    index = 1
11    while True:
12        try:
13            keras_layer = keras_model.get_layer(f"lstm_{index}")
14        except ValueError:
15            break
16        layer = LSTM()
17        layer.load_keras_weights(keras_layer.get_weights())
18        recurrent_layers.append(layer)
19        index += 1
20
21    output_dense = Dense()
22    output_dense.load_keras_weights(keras_model.get_layer("word_logits").get_weights())
23    return cls(feature_projection, embedding, recurrent_layers, output_dense)
```

```

1 def forward(self, image_features: np.ndarray, caption_input_ids: np.ndarray) -> np.ndarray:
2     features = np.asarray(image_features, dtype=np.float32)
3     if features.ndim == 1:
4         features = features[np.newaxis, :]
5
6     token_ids = np.asarray(caption_input_ids, dtype=np.int32)
7     if token_ids.ndim == 1:
8         token_ids = token_ids[np.newaxis, :]
9
10    feature_timestep = self.feature_projection.forward(features)[: , np.newaxis, :]
11    token_embeddings = self.embedding.forward(token_ids)
12    sequence = np.concatenate([feature_timestep, token_embeddings], axis=1)
13
14    x = sequence
15    for layer in self.recurrent_layers:
16        x = layer.forward(x)
17
18    token_outputs = x[:, 1:, :]
19    logits = self.output_dense.forward(token_outputs)
20    return softmax(logits, axis=-1)

```

6. CNN Configuration dan Training

a. Class SharedConvCNNConfig (src/tubes2_ml/cnn/models.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
input_shape	tuple[int, int, int]	Shape input gambar, default (96, 96, 3).
num_classes	int	Jumlah kelas Intel Image Classification, default 6.
conv_filters	list[LSTM]	Banyak filter tiap layer Conv2D.
kernel_sizes	Dense	Ukuran kernel tiap Conv2D.
pooling_type	str	Jenis pooling, max atau average.
dense_units	tuple[int, ...]	Unit pada Dense hidden head.
dropout_rate	float	Dropout setelah Dense hidden.
learning_rate	float	Learning rate Adam.
activation	str	Aktivasi Conv2D dan Dense hidden.
name	str	Nama model/eksperimen.
compile_model	bool	Menentukan apakah model

		langsung di-compile.
metrics	tuple[str, ...]	Metrik Keras saat training.
Deskripsi: Dataclass konfigurasi untuk membangun CNN shared-parameter menggunakan Keras.		

b. Class CNNTrainingConfig (src/tubes2_ml/cnn/train.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
train_dir	str Path	Folder data training Intel.
validation_dir	str Path None	Folder validation eksplisit; jika None, memakai validation split.
output_dir	str Path	Folder output model dan bobot.
history_dir	str Path	Folder output history dan metadata.
image_size	tuple[int, int]	Ukuran resize gambar.
batch_size	int	Ukuran batch training.
epochs	int	Jumlah epoch.
validation_split	float	Rasio validation dari train directory.
seed	int	Seed random.
save_format	str	Format penyimpanan model.
early_stopping_patience	int None	Patience early stopping.
Deskripsi: Dataclass konfigurasi training CNN Keras.		

7. Captioning Configuration, Data, dan Training

a. Class CaptionDecoderConfig (src/tubes2_ml/captioning/models.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
--------------	------	-----------

vocab_size	int	Jumlah token vocabulary.
feature_dim	int	Dimensi feature vector CNN encoder.
max_caption_length	int	Panjang maksimum sequence caption.
embed_dim	int	Dimensi embedding token dan proyeksi image feature.
hidden_units	int	Ukuran hidden state RNN/LSTM.
num_recurrent_layers	int	Jumlah layer recurrent.
dropout_rate	float	Dropout recurrent layer.
learning_rate	float	Learning rate Adam.
decoder_type	Literal["pre", "init"]	Jenis decoder.
injection_mode	Literal["pre", "init"]	Mode injeksi image feature. Spek utama memakai pre.
name	str	Nama model.
Deskripsi: Dataclass konfigurasi decoder captioning Keras untuk RNN/LSTM.		

b. Class CaptionTrainingConfig (src/tubes2_ml/captioning/train.py)

Atribut

Nama_Atribut	Tipe	Deskripsi
processed_dir	Path	Folder hasil preprocessing caption.
features_dir	Path	Folder feature vector CNN encoder.
output_dir	Path	Folder output model Keras.
history_dir	Path	Folder output history dan metadata.
batch_size	int	Ukuran batch training.
epochs	int	Jumlah epoch.

seed	int	Seed random.
Deskripsi: Konfigurasi proses training decoder captioning.		

c. **Class CaptionTrainingResult (src/tubes2_ml/captioning/train.py)**

Atribut

Nama_Atribut	Tipe	Deskripsi
model_path	Path	Path model .keras hasil training.
history_path	Path	Path file history JSON.
metadata_path	Path	Path file metadata JSON.
decoder_type	str	Jenis decoder, rnn atau lstm.
injection_mode	str	Mode injeksi feature.
num_recurrent_layers	int	Jumlah recurrent layer.
hidden_units	int	Ukuran hidden state.
best_validation_loss	float	Validation loss terbaik.
Deskripsi: Dataclass hasil training satu variasi decoder captioning.		

d. **Class FeatureExtractionResult (src/tubes2_ml/captioning/feature_extraction.py)**

Atribut

Nama_Atribut	Tipe	Deskripsi
features_path	Path	Path file features.npy.
image_ids_path	Path	Path file image_ids.json.
metadata_path	Path	Path metadata ekstraksi.
num_images	int	Jumlah gambar yang diekstraksi.
feature_shape	tuple[int, ...]	Shape feature per gambar.
encoder_name	str	Nama encoder yang digunakan.

hidden_units	int	Ukuran hidden state.
best_validation_loss	float	Validation loss terbaik.
Deskripsi: Dataclass ringkasan hasil ekstraksi feature.		

e. **Class CaptionVocabulary (src/tubes2_ml/captioning/decoding.py)**

Atribut

Nama_Atribut	Tipe	Deskripsi
word_to_id	dict[str, int]	Mapping token ke id.
id_to_word	dict[str, int]	Mapping id ke token.
pad_token	str	Token padding.
start_token	str	Token awal caption.
end_token	str	Token akhir caption.
unk_token	str	Token unknown.
pad_id	int	Property id token padding.
start_id	int	Property id token start.
end_id	int	Property id token end.

Method

Nama_Atribut	Tipe	Deskripsi
pad_id	Property	-
start_id	Property	-
end_id	Property	-
ids_to_words	Property	token_ids, skip_special
ids_to_caption	Property	token_idx
Deskripsi: Menyimpan vocabulary captioning dan menyediakan method untuk mengubah token id hasil decoder menjadi kata atau caption string.		

8. Utility Classes

a. Class Timer

Atribut

Nama Atribut	Tipe	Parameter
durations	dict[str, float]	Menyimpan durasi eksekusi untuk nama blok tertentu.

Method

Nama Atribut	Tipe	Parameter
track	Context Manager Method	name: str
Deskripsi: Mencatat durasi eksekusi blok kode menggunakan context manager.		

b. Class MetricsPayload

Atribut

Nama Atribut	Tipe	Parameter
experiment	str	Nama eksperimen.
metrics	dict[str, float]	Nilai metrik eksperimen.
metadata	dict[str, Any] None	Metadata tambahan.
Deskripsi: Dataclass untuk menyimpan payload metrik eksperimen ke JSON.		

B. Penjelasan Forward Propagation

Forward propagation adalah proses menghitung output model dari input dengan melewati data secara berurutan melalui layer-layer jaringan. Pada repository ini, forward propagation diimplementasikan untuk:

- CNN untuk klasifikasi gambar.
- RNN untuk image captioning.
- LSTM untuk image captioning.

Penjelasan di bawah menghubungkan konsep matematis, arti simbol, dan kode implementasi yang digunakan.

Konsep Dasar:

Forward pass menghitung aktivasi layer demi layer:

```
input -> layer 1 -> layer 2 -> ... -> output
```

Untuk layer umum, notasinya:

$$\begin{aligned} z^{(l)} &= f^{(l)}(a^{(l-1)}, W^{(l)}, b^{(l)}) \\ a^{(l)} &= g(z^{(l)}) \end{aligned}$$

Keterangan:

Simbol	Arti
$a^{(l-1)}$	Aktivasi/output dari layer sebelumnya.
$W^{(l)}$	Bobot layer ke-l
$b^{(l)}$	Bias layer ke-l
$z^{(l)}$	Pre-activation, hasil operasi linear/konvolusi sebelum aktivasi
$g()$	Fungsi aktivasi seperti ReLU, tanh, sigmoid, atau softmax.
$a^{(l)}$	Output layer ke-l

Pola ini, dapat dilihat pada file [dense.py](#)

```
output = x @ self.weights
if self.bias is not None:
    output += self.bias

return self.activation(output)
```

Kode tersebut sesuai dengan rumus:

$$\begin{aligned} z &= xW + b \\ a &= g(z) \end{aligned}$$

1. CNN

CNN digunakan untuk task image classification pada dataset Intel Image Classification. Forward CNN terdiri dari operasi konvolusi, aktivasi, pooling, global pooling/flatten, dense, dan softmax.

1.1 Alur Forward Propagation

Pada model Keras CNN, alurnya adalah:

```
Input image
→ Conv2D
→ ReLU
→ Max/Average Pooling
→ Conv2D berikutnya, jika ada
→ GlobalAveragePooling2D
→ Dense
→ Dropout
→ Dense output
→ Softmax
→ Probabilitas kelas
```

Implementasi model Keras:

```
def _add_conv_block(x, filters: int, kernel_size: int,
pooling_type: str, activation: str, index: int):
    x = tf.keras.layers.Conv2D(
        filters=filters,
        kernel_size=(kernel_size, kernel_size),
        padding="same",
        activation=activation,
        name=f"conv_{index}",
    )(x)

    pooling_layer = (
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2),
name=f"pool_{index}")
        if pooling_type == "max"
        else tf.keras.layers.AveragePooling2D(pool_size=(2, 2),
name=f"pool_{index}")
    )

    return pooling_layer(x)
```

Head Klasifikasi:

```
def _add_dense_head(x, config: SharedConvCNNConfig):
```

```

x =
tf.keras.layers.GlobalAveragePooling2D(name="global_average_pool"
)(x)

for dense_index, units in enumerate(config.dense_units,
start=1):
    x = tf.keras.layers.Dense(units,
activation=config.activation, name=f"dense_{dense_index}") (x)
    if config.dropout_rate > 0:
        x = tf.keras.layers.Dropout(config.dropout_rate,
name=f"dropout_{dense_index}") (x)

return tf.keras.layers.Dense(
    config.num_classes,
    activation="softmax",
    name="class_probabilities",
) (x)

```

1.2 Forward Conv2D Shared Parameter

Conv2D menghitung dot product antara patch input dan kernel. Karena memakai shared parameter, kernel yang sama digunakan pada semua posisi gambar.

Rumus untuk satu output pixel:

$$Y[n, i, j, k] = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} \sum_{c=0}^{C_{in}-1} X[n, i + u, j + v, c] \cdot W[u, v, c, k] + b[k]$$

Keterangan:

Simbol	Arti
X	Input image/batch feature map.
Y	Output feature map.
n	Index batch.
i,j	Posisi spasial output.
u,v	Posisi di dalam kernel.
c	Channel input.
k	Channel/filter output.
W	Kernel Conv2D dengan shape (kH, kW, C_in, C_out).
b	Bias per filter output.

Kode yang sesuai:

```
for row in range(output_height):
    row_start = row * stride_h
    row_end = row_start + kernel_height
    for col in range(output_width):
        col_start = col * stride_w
        col_end = col_start + kernel_width
        patch = padded[:, row_start:row_end, col_start:col_end,
:]
        output[:, row, col, :] = np.tensordot(
            patch,
            self.kernel,
            axes=((1, 2, 3), (0, 1, 2)),
        )

if self.bias is not None:
    output += self.bias.reshape(1, 1, 1, output_channels)

return self.activation(output)
```

Hubungan kode dengan rumus:

Rumus	Kode
$X[n, i + u, j + v, c]$	patch
$W[u, v, c, k]$	self.kernel
$sum(...)$	np.tensordot(...)
$+ b(k)$	Output += self.bias.reshape(...)
$g\{Y\}$	return self.activation(output)

Contoh shape:

```
Input X      : (64, 96, 96, 3)
Kernel W     : (3, 3, 3, 32)
Bias b       : (32,)
Output Y same : (64, 96, 96, 32)
```

Jika memakai pooling 2x2 setelah Conv2D:

```
Output pooling: (64, 48, 48, 32)
```

1.3 Forward LocallyConnected2D Non-Shared Parameter

LocallyConnected2D mirip Conv2D, tetapi tidak memakai parameter sharing. Setiap posisi output punya kernel sendiri.

Rumus:

$$Y[n, p, k] = \sum_{q=0}^{K_h K_w C_{in} - 1} P[n, p, q] \cdot W[p, q, k] + b[p, k]$$

Keterangan:

Simbol	Arti
p	Index posisi output, hasil flatten dari (i, j).
q	Index elemen patch yang sudah diratakan.
P	Patch input yang sudah diratakan.
W[p]	Kernel khusus untuk posisi output ke-p.
W[p]	Bias untuk posisi output ke-p

Kode yang sesuai:

```
position = 0
for row in range(output_height):
    row_start = row * stride_h
    row_end = row_start + kernel_height
    for col in range(output_width):
        col_start = col * stride_w
        col_end = col_start + kernel_width
        patch = padded[:, row_start:row_end, col_start:col_end,
:].reshape(batch_size, -1)
        output[:, row, col, :] = patch @ self.kernel[position]
        position += 1

if self.bias is not None:
    output += self._reshape_bias(self.bias, output_height,
output_width, output_channels)

return self.activation(output)
```

Hubungan kode dengan rumus:

Rumus	Kode
P[n, p, q]	patch
W[p, q, k]	self.kernel(position)
sum_q P * W	patch @ self.kernel(position)

b[p, k]	self._reshape_bias(...)
---------	-------------------------

Perbandingan:

Layer	Parameter
Conv2D	Satu kernel dipakai ulang di seluruh posisi.
LocallyConnected2D	Tiap posisi output punya kernel sendiri.

Karena itu LocallyConnected2D jauh lebih mahal secara parameter.

1.4 Forward Pooling

Pooling mereduksi ukuran spasial feature map. Ada dua mode:

1. Max pooling

$$Y[n, i, j, c] = \max_{(u, v) \in R} (X[n, i + u, j + v, c])$$

2. Average pooling

$$Y[n, i, j, c] = \frac{1}{|R|} \sum_{(u, v) \in R} X[n, i + u, j + v, c]$$

Keterangan:

Simbol	Arti
R	Area/window pooling
C	Channel feature map
,	R

Kode:

```
patch = padded[:, row_start:row_end, col_start:col_end, :]
if self.mode == "max":
    output[:, row, col, :] = np.max(patch, axis=(1, 2))
else:
    output[:, row, col, :] = np.mean(patch, axis=(1, 2))
```

Pooling dilakukan independen untuk setiap channel.

1.5 Forward Global Average Pooling

GlobalAveragePooling2D mengubah feature map (N, H, W, C) menjadi (N, C).

Rumus:

$$Y[n, c] = \frac{1}{H \cdot W} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} X[n, i, j, c]$$

Kode:

```
return np.mean(x, axis=(1, 2))
```

Hubungan:

Rumus	Kode
sum over H and W	axis=(1,2)
dibagi H * W	np.mean

1.6 Forward Dense dan Softmax pada CNN

Dense layer:

$$z = xW + b$$

Aktivasi:

$$a = g(z)$$

Kode:

```
output = x @ self.weights
if self.bias is not None:
    output += self.bias
return self.activation(output)
```

Untuk output klasifikasi, aktivasi softmax:

$$y_k = \frac{e^{z_k}}{\sum_{r=1}^C e^{z_r}}$$

Kode softmax:

```
def softmax(x: np.ndarray, axis: int =
-1) -> np.ndarray:
    shifted = x - np.max(x, axis=axis,
keepdims=True)
    exp = np.exp(shifted)
    return exp / np.sum(exp, axis=axis,
keepdims=True)
```

Output akhir CNN:

```
probabilities.shape = (batch_size, 6)
```

Setiap nilai menunjukkan probabilitas kelas buildings, forest, glacier, mountain, sea, atau street.

2. RNN

RNN digunakan sebagai decoder image captioning. Arsitektur utama memakai metode pre-inject, yaitu feature CNN dimasukkan sebagai timestep pertama sebelum token <start>.

2.1 Konsep Forward RNN Captioning

Alur:

```
image
→ CNN encoder frozen
→ feature vector
→ Dense projection
→ x_{-1}
→ Embedding caption input
→ SimpleRNN
→ Dense output
→ Softmax vocabulary
```

Input training:

```
[CNN_feature, <start>, S0, S1, ..., S_{N-1}]
```

Target:

```
[S0, S1, ..., S_N]
```

2.2 Dense Projection untuk Feature CNN

Feature CNN dari InceptionV3 memiliki shape:

```
image_feature.shape = (batch, 2048)
```

Feature ini diproyeksikan ke embed_dim:

$$x_{-1} = W_f v + b_f$$

Keterangan:

Simbol	Arti
v	Feature vector CNN
W_f	Bobot Dense projection.
b_f	Bias Dense projection.
X_{-1}	Feature timestep sebelum token <start>.

Kode Keras:

```
feature_embedding = tf.keras.layers.Dense(config.embed_dim,
name="feature_projection")(feature_input)
feature_embedding = tf.keras.layers.Reshape((1,
config.embed_dim), name="feature_timestep")(feature_embedding)
```

Kode Scratch:

```
feature_timestep = self.feature_projection.forward(features)[: ,
np.newaxis, :]
```

Jika:

```
features.shape = (batch, 2048)
embed_dim = 256
```

Maka:

```
feature_timestep.shape = (batch, 1, 256)
```

2.3 Embedding Token Caption

Token id caption diubah menjadi vektor embedding.

Rumus:

$$x_t = E[w_t]$$

Keterangan:

Simbol	Arti
W_t	Token id pada timestep t.
E	Matriks embedding dengan shape (vocab_size, embed_dim).
X_t	Vektor embedding token.

Kode:

```
ids = np.asarray(token_ids, dtype=np.int64)
return self.weights[ids]
```

Jika:

```
token_ids.shape = (batch, max_caption_length)
embedding.shape = (vocab_size, 256)
```

Maka:

```
token_embeddings.shape = (batch, max_caption_length, 256)
```

2.4 Penggabungan Feature Timestep dan Token Embedding

Pada pre-inject, feature timestep ditempel di depan sequence token:

$$X = [X_{-1}, X_0, \dots, X_t]$$

Kode Keras:

```
sequence = tf.keras.layers.Concatenate(axis=1,
name="preinject_sequence")(
    [feature_embedding, token_embedding]
)
```

Kode scratch:

```
sequence = np.concatenate([feature_timestep, token_embeddings], axis=1)
```

Contoh Shape

```
feature_timestep.shape = (batch, 1, 256)
```

```
token_embeddings.shape = (batch, 38, 256)
sequence.shape = (batch, 39, 256)
```

2.5 Forward SimpleRNN Cell

SimpleRNN menyimpan satu hidden state h_t . Rumus forward:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

Keterangan:

Simbol	Arti
X_t	Input pada timestep ke-t.
W_x	Bobot input-to-hidden (kernel).
h_{t-1}	Hidden state timestep sebelumnya.
W_h	Bobot hidden-to-hidden (recurrent_kernel).
b	Bias.
h_t	Hidden state baru.

Kode:

```
for timestep in range(timesteps):
    h_t = tanh(inputs[:, timestep, :] @ self.kernel + h_t @
self.recurrent_kernel + bias)
    outputs.append(h_t)
```

Hubungan rumus dan kode:

Rumus	Kode
X_t	inputs[:, timestep, :]
W_x	self.kernel
h_{t-1}	h_t sebelum update
W_h	self.recurrent_kernel

b	bias
$\tanh(\dots)$	$\tanh(\dots)$

Initial state:

```
if initial_state is None:
    h_t = np.zeros((batch_size, self.units), dtype=np.float32)
```

Artinya h_0 bernilai nol.

2.6 Output RNN ke Vocabulary

Setelah sequence diproses RNN, output timestep feature dibuang:

```
token_outputs = x[:, 1:, :]
```

Alasannya, timestep pertama adalah image feature X_{-1} , sedangkan target caption dimulai dari kata pertama.

Logits vocabulary:

$$z_t = h_t W_o + b_o$$

Probabilitas vocabulary:

$$P(w_t = k) = \frac{e^{z_k}}{\sum_{r=1}^V e^{z_{l_r}}}$$

Kode:

```
logits = self.output_dense.forward(token_outputs)
return softmax(logits, axis=-1)
```

Jika:

```
token_outputs.shape = (batch, 38, hidden_units)
vocab_size = 7464
```

Maka:

```
probabilities.shape = (batch, 38, 7464)
```

3. LSTM

LSTM digunakan sebagai decoder captioning seperti RNN, tetapi memiliki hidden state ht dan cell state ct

3.1 Konsep Dasar LSTM

LSTM dirancang untuk mengatasi masalah vanishing gradient pada RNN biasa. Cell state c_t berperan sebagai jalur memori yang dapat mempertahankan informasi lebih lama.

Forward LSTM memiliki empat komponen utama:

1. Input gate.
2. Forget gate.
3. Candidate cell.
4. Output gate.

3.2 Format Bobot LSTM Keras

Keras menyimpan bobot LSTM dalam tiga array:

```
kernel
recurrent_kernel
bias
```

Shape:

```
kernel.shape = (input_dim, 4 * units)
recurrent_kernel.shape = (units, 4 * units)
bias.shape = (4 * units,)
```

Kode load bobot:

```
def load_keras_weights(self, weights: list[np.ndarray] |
tuple[np.ndarray, ...]) -> None:
    if len(weights) != 3:
        raise ValueError("LSTM weights must be [kernel,
recurrent_kernel, bias]")
    self.kernel = np.asarray(weights[0],
dtype=np.float32)
    self.recurrent_kernel = np.asarray(weights[1],
dtype=np.float32)
    self.bias = np.asarray(weights[2], dtype=np.float32)
```

Dimensi $4 * units$ kemudian dipisah menjadi:

- Input gate
- Forget gate
- Candidate cell
- Output gate

Rumus Forward LSTM:

Pertama, hitung gabungan pre-activation:

$$z_t = x_t W_x + h_{t-1} W_h + b$$

Kemudian split:

$$z_t = [z_i, z_f, z_c, z_o]$$

Gate:

$$\begin{aligned} i_t &= \sigma(z_i) \\ f_t &= \sigma(z_f) \\ c_t^* &= \tanh(z_c) \\ o_t &= \sigma(z_o) \end{aligned}$$

Update cell state:

$$c_t = f_t \odot C_{t-1} + I_t \odot C_t^*$$

Update hidden state:

$$h_t = o_t \odot \tanh(c_t)$$

Keterangan:

Simbol	Arti
I_t	Input gate, mengatur informasi baru yang masuk.
f_t	Forget gate, mengatur informasi lama yang dipertahankan.
c_t^*	Candidate cell, kandidat memori baru.
O_t	Output gate, mengatur informasi yang keluar ke hidden state.

C_t	Cell state/memori internal.
h_t	Hidden state/output timestep.
$\odot$	Perkalian elemen-wise.

3.3 Kode Forward LSTM

Kode Implementasi:

```
for timestep in range(timesteps):
    z = inputs[:, timestep, :] @ self.kernel + h_t @
self.recurrent_kernel + bias
    z_i, z_f, z_c, z_o = np.split(z, 4, axis=-1)
    i_t = sigmoid(z_i)
    f_t = sigmoid(z_f)
    c_hat_t = tanh(z_c)
    o_t = sigmoid(z_o)
    c_t = f_t * c_t + i_t * c_hat_t
    h_t = o_t * tanh(c_t)
    outputs.append(h_t)
```

Initial state:

```
if initial_state is None:
    h_t = np.zeros((batch_size, self.units), dtype=np.float32)
    c_t = np.zeros((batch_size, self.units), dtype=np.float32)
```

Artinya h_0 dan c_0 bernilai nol.

3.4 Forward LSTM Captioner

Alur LSTM captioner sama seperti RNN captioner:

```
image_features
→ Dense projection
→ feature timestep
→ caption_input_ids
→ Embedding
→ concatenate
→ LSTM layer(s)
→ remove feature timestep
→ Dense output
→ softmax
```

Kode:

```
feature_timestep = self.feature_projection.forward(features)[:,
```

```

np.newaxis, :]
token_embeddings = self.embedding.forward(token_ids)
sequence = np.concatenate([feature_timestep, token_embeddings],
axis=1)

x = sequence
for layer in self.recurrent_layers:
    x = layer.forward(x)

token_outputs = x[:, 1:, :]
logits = self.output_dense.forward(token_outputs)
return softmax(logits, axis=-1)

```

Jika menggunakan LSTM, `self.recurrent_layers` berisi objek LSTM. Jika menggunakan RNN, `self.recurrent_layers` berisi objek SimpleRNN.

Forward Inference Caption Generation

Setelah model menghasilkan probabilitas vocabulary, caption dibuat dengan decoding.

1. Greedy Decoding

Greedy decoding memilih token dengan probabilitas terbesar pada setiap timestep.

Rumus:

$$w_t = \operatorname{argmax}(k) P(w_t = k | w_{<t}, \text{image})$$

Kode:

```

for position in range(max_caption_length):
    model_input = make_padded_input(prefix,
max_caption_length, vocabulary.pad_id)
    probabilities = predict_probs(np.asarray(image_feature,
dtype=np.float32)[np.newaxis, :], model_input)
    next_id = int(np.argmax(probabilities[0, position]))
    if next_id == vocabulary.end_id:
        break
    generated.append(next_id)
    prefix.append(next_id)

```

Penjelasan:

1. Prefix dimulai dengan token <start>.
2. Model memprediksi probabilitas vocabulary.
3. Token dengan probabilitas terbesar dipilih.
4. Token tersebut ditambahkan ke prefix.
5. Proses berhenti jika token <end> muncul.

2. Beam Search

Beam search menyimpan beberapa kandidat caption terbaik, bukan hanya satu kandidat seperti greedy decoding.

Skor Beam:

$$score = \sum_t \log P(w_t)$$

Kode:

```
top_ids = np.argsort(probabilities)[-beam_width:][::-1]
for token_id in top_ids:
    token_id = int(token_id)
    probability = max(float(probabilities[token_id]),
1e-12)
    next_prefix = prefix + [token_id]
    candidates.append((next_prefix, score +
math.log(probability)
, token_id == vocabulary.end_id))
```

Kandidat terbaik dipilih dengan normalisasi panjang:

```
beams = sorted(
    candidates,
    key=lambda item: item[1] / max(1, len(item[0]) - 1),
    reverse=True,
)[:beam_width]
```

C. Hasil Pengujian

1. CNN

Model terbaik berdasarkan validation macro F1 adalah:

Model	Validation Macro-F1
cnn_2conv_f32-64_k5-5_maxpool	0.6638

Metrik Macro F1

Macro F1 menghitung F1-score untuk setiap kelas, lalu mengambil rata-ratanya.

```
Precision_c = TP_c / (TP_c + FP_c)
Recall_c     = TP_c / (TP_c + FN_c)
```

$$\begin{aligned}
 F1_c &= 2 * Precision_c * Recall_c / (Precision_c + Recall_c) \\
 Macro\ F1 &= (1 / C) * sum(F1_c)
 \end{aligned}$$

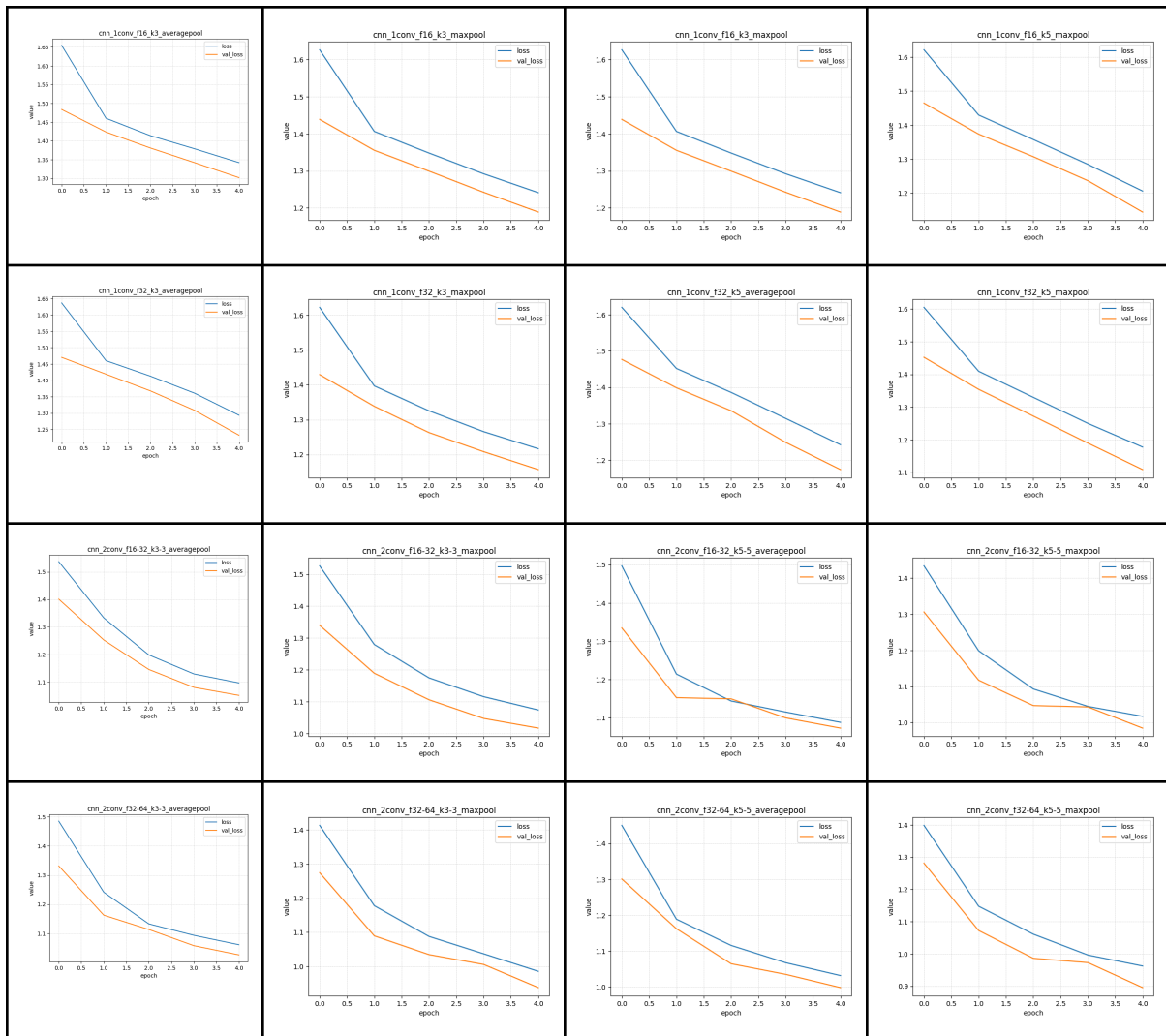
Macro F1 cocok untuk klasifikasi multi-kelas karena setiap kelas diberi bobot yang sama, tidak hanya kelas mayoritas.

Hasil 16 Variasi Hyperparameter

Tabel berikut menunjukkan hasil akhir setiap arsitektur berdasarkan validation macro F1. Kolom loss memperlihatkan train loss dan validation loss awal/akhir dari history training.

Model	Conv	Filter	Kernel	Pool	Val Macro F1	Train Loss Awal → Akhir	Val Loss Awal → Akhir
cnn_2conv_f32-64_k5-5_maxpool	2	32-64	5-5	max	0.6638	1.399 -> 0.962	1.281 -> 0.895
cnn_2conv_f32-64_k3-3_maxpool	2	32-64	3-3	max	0.6329	1.413 -> 0.985	1.274 -> 0.938
cnn_2conv_f16-32_k5-5_maxpool	2	16-32	5-5	max	0.6078	1.434 -> 1.017	1.306 -> 0.985
cnn_2conv_f32-64_k5-5_averagepool	2	32-64	5-5	average	0.6022	1.451 -> 1.031	1.301 -> 0.998
cnn_2conv_f32-64_k3-3_averagepool	2	32-64	3-3	average	0.5868	1.485 -> 1.061	1.332 -> 1.026
cnn_2conv_f16-32_k3-3_maxpool	2	16-32	3-3	max	0.5860	1.527 -> 1.074	1.340 -> 1.017
cnn_2conv_f16-32_k5-5_averagepool	2	16-32	5-5	average	0.5839	1.497 -> 1.088	1.335 -> 1.073
cnn_2conv_f16-32_k3-3_averagepool	2	16-32	3-3	average	0.5788	1.537 -> 1.096	1.401 -> 1.051
cnn_1conv_f16_k5_averagepool	1	16	5	average	0.5259	1.649 -> 1.199	1.491 -> 1.148
cnn_1conv_f16_k5_maxpool	1	16	5	max	0.5171	1.622 -> 1.206	1.465 -> 1.144

cnn_1conv_f32_k 5_maxpool	1	32	5	max	0.5078	1.605 -> 1.177	1.452 -> 1.108
cnn_1conv_f32_k 5_averagepool	1	32	5	average	0.5050	1.620 -> 1.242	1.477 -> 1.174
cnn_1conv_f16_k 3_maxpool	1	16	3	max	0.5007	1.627 -> 1.241	1.439 -> 1.189
cnn_1conv_f32_k 3_maxpool	1	32	3	max	0.4855	1.622 -> 1.217	1.429 -> 1.157
cnn_1conv_f32_k 3_averagepool	1	32	3	average	0.4643	1.637 -> 1.293	1.470 -> 1.232
cnn_1conv_f16_k 3_averagepool	1	16	3	average	0.4033	1.654 -> 1.342	1.484 -> 1.302



Ringkasan Loss Akhir per Hyperparameter

Selain membandingkan validation macro F1, eksperimen juga dianalisis berdasarkan training loss akhir dan validation loss akhir. Nilai pada tabel berikut merupakan rata-rata dari seluruh run yang termasuk ke dalam masing-masing kelompok hyperparameter.

1. Berdasarkan Jumlah Layer Konvolusi

Jumlah Conv Layer	Jumlah Run	Avg Final Train Loss	Avg Final Val Loss	Avg Val Macro F1
1	8	1.2397	1.1817	0.4887
2	8	1.0394	0.9978	0.6053

Berdasarkan tabel di atas, model dengan 2 convolution layer tidak hanya memiliki nilai macro F1 yang lebih tinggi, tetapi juga menghasilkan training loss akhir dan validation loss akhir yang lebih rendah dibandingkan model dengan 1 convolution layer. Hal ini memperkuat kesimpulan bahwa penambahan layer konvolusi membantu model mempelajari pola visual dengan lebih baik, bukan hanya meningkatkan skor F1 secara kebetulan.

2. Berdasarkan Jumlah Filter

Filter	Jumlah Run	Avg Final Train Loss	Avg Final Val Loss	Avg Val Macro F1
16	4	1.2471	1.1956	0.4868
32	4	1.2323	1.1678	0.4907
16-32	4	1.0689	1.0314	0.5891
32-64	4	1.0099	0.9642	0.6215

Berdasarkan hasil tersebut, kombinasi filter bertingkat, terutama 32-64, memberikan loss akhir paling rendah dan macro F1 paling tinggi. Artinya, peningkatan kapasitas filter membantu model mengurangi error selama training sekaligus memperbaiki generalisasi pada validation set.

3. Berdasarkan Ukuran Kernel

Kernel	Jumlah Run	Avg Final Train Loss	Avg Final Val Loss	Avg Val Macro F1
3	4	1.2733	1.2199	0.4635
5	4	1.2062	1.1434	0.5140
3-3	4	1.0541	1.0080	0.5961

5-5	4	1.0247	0.9876	0.6144
-----	---	--------	--------	--------

Kernel 5-5 menghasilkan training loss akhir dan validation loss akhir paling rendah. Hasil ini konsisten dengan nilai macro F1, sehingga penggunaan kernel yang lebih besar pada eksperimen ini tidak hanya meningkatkan metrik klasifikasi, tetapi juga membantu model mencapai loss yang lebih baik.

4. Berdasarkan Jenis Pooling

Pooling	Jumlah Run	Avg Final Train Loss	Avg Final Val Loss	Avg Val Macro F1
Average	8	1.1692	1.1255	0.5313
Max	8	1.1099	1.0540	0.5627

Max pooling menghasilkan training loss akhir dan validation loss akhir yang lebih rendah dibandingkan average pooling. Pola ini juga sejalan dengan nilai macro F1 yang lebih tinggi, sehingga max pooling dapat disimpulkan lebih efektif pada dataset ini karena mampu mempertahankan aktivasi fitur yang paling kuat.

A. Pengaruh Jumlah Layer Konvolusi

Ringkasan berdasarkan jumlah layer:

Jumlah Conv Layer	Mean Validation Macro F1	Best Validation Macro F1	Jumlah Run
1	0.4887	0.5259	8
2	0.6053	0.6638	8

Berdasarkan hasil eksperimen, jumlah layer konvolusi memberi pengaruh yang cukup besar terhadap performa model. Model dengan 2 convolution layer menghasilkan mean validation macro F1 sebesar 0.6053, sedangkan model dengan 1 convolution layer hanya mencapai 0.4887. Nilai terbaiknya juga meningkat dari 0.5259 pada model 1 conv menjadi 0.6638 pada model 2 conv. Kenaikan ini menunjukkan bahwa tambahan layer konvolusi membantu model membangun representasi visual yang lebih kaya.

Secara konsep, layer konvolusi pertama biasanya menangkap pola sederhana seperti edge, garis, warna dominan, dan tekstur lokal. Setelah itu, layer konvolusi berikutnya dapat mengombinasikan pola-pola sederhana tersebut menjadi fitur yang lebih kompleks, misalnya bentuk bangunan, area pepohonan, permukaan laut, jalan, atau tekstur pegunungan. Karena dataset Intel Image Classification memiliki kelas dengan perbedaan visual yang cukup kompleks, arsitektur 2 conv layer menjadi lebih efektif dibanding arsitektur 1 conv layer.

Kesimpulan:

Menambah jumlah conv layer dari 1 ke 2 meningkatkan kemampuan representasi visual dan berdampak positif pada macro F1.

B. Pengaruh Jumlah Filter

Ringkasan variasi filter:

Filter	Mean Validation Macro F1	Best Validation Macro F1	Jumlah Run
16	0.4868	0.5259	4
32	0.4907	0.5078	4
16-32	0.5891	0.6078	4
32-64	0.6215	0.6638	4

Jumlah filter juga memengaruhi kapasitas representasi model. Pada eksperimen ini, kombinasi filter bertingkat 32-64 memberikan hasil terbaik dengan mean validation macro F1 sebesar 0.6215 dan best validation macro F1 sebesar 0.6638. Kombinasi 16-32 juga sudah lebih baik daripada model 1 conv, tetapi masih berada di bawah 32-64. Hal ini menunjukkan bahwa penambahan filter memberi model ruang yang lebih besar untuk mempelajari variasi visual dari masing-masing kelas.

Secara intuitif, semakin banyak filter berarti semakin banyak jenis pola yang dapat dipelajari oleh layer konvolusi. Misalnya, sebagian filter dapat sensitif terhadap tekstur hutan, sebagian lain terhadap garis bangunan, permukaan air laut, atau kontur pegunungan. Namun jumlah filter yang lebih besar juga membuat jumlah parameter dan biaya komputasi meningkat. Karena itu, peningkatan filter perlu diseimbangkan dengan ukuran dataset, waktu training, dan risiko overfitting. Pada eksperimen ini, 32-64 masih menjadi titik yang paling baik karena performanya meningkat tanpa terlihat kalah pada validation macro F1.

Kesimpulan:

Kombinasi filter bertingkat 32-64 paling efektif pada eksperimen ini karena memberi kapasitas representasi yang lebih kuat.

C. Pengaruh Ukuran Kernel

Ringkasan variasi kernel:

Kernel	Mean Validation Macro F1	Best Validation Macro F1	Jumlah Run
--------	--------------------------	--------------------------	------------

3	0.4635	0.5007	4
3	0.5140	0.5259	4
3-3	0.5961	0.6329	4
5-5	0.6144	0.6638	4

Ukuran kernel berpengaruh terhadap seberapa luas konteks lokal yang dilihat model pada satu operasi konvolusi. Pada model 1 conv, kernel 5 memberikan mean validation macro F1 sebesar 0.5140, lebih tinggi dibanding kernel 3 yang hanya mencapai 0.4635. Pola yang sama juga terlihat pada model 2 conv, di mana kombinasi 5-5 menghasilkan mean validation macro F1 sebesar 0.6144 dan best score 0.6638, lebih tinggi dibanding 3-3.

Kernel yang lebih besar memiliki receptive field yang lebih luas, sehingga model dapat menangkap pola spasial yang sedikit lebih besar dalam satu langkah. Untuk dataset seperti Intel Image Classification, informasi visual penting sering muncul sebagai area atau struktur yang tidak terlalu kecil, misalnya permukaan laut, massa hutan, lereng gunung, atau susunan bangunan. Dengan input berukuran 96x96, kernel 5 masih cukup ringan secara komputasi, tetapi memberi konteks lokal yang lebih luas daripada kernel 3. Inilah kemungkinan alasan kernel 5-5 menjadi konfigurasi terbaik pada eksperimen ini.

Kesimpulan:

Ukuran kernel 5 memberikan hasil terbaik pada eksperimen ini karena membantu model menangkap konteks visual lokal yang lebih luas.

D. Pengaruh Jenis Pooling

Ringkasan variasi pooling:

Pooling	Mean Validation Macro F1	Best Validation Macro F1	Jumlah Run
Average Pooling	0.5313	0.6022	8
Max Pooling	0.5627	0.6638	8

Jenis pooling juga memberi pengaruh terhadap performa akhir model. Max pooling menghasilkan mean validation macro F1 sebesar 0.5627 dan best validation macro F1 sebesar 0.6638, lebih tinggi dibanding average pooling yang memiliki mean 0.5313 dan best 0.6022. Hasil ini menunjukkan bahwa mempertahankan aktivasi terkuat pada setiap window lebih cocok untuk task klasifikasi gambar ini.

Max pooling bekerja dengan mengambil nilai maksimum dari suatu area kecil pada feature map. Dengan cara ini, fitur yang paling dominan dan diskriminatif tetap dipertahankan, misalnya edge bangunan, garis jalan, tekstur hutan, atau batas antara laut dan langit. Sebaliknya, average pooling mengambil rata-rata aktivasi sehingga sinyal kuat dapat menjadi lebih halus atau melemah. Untuk klasifikasi, terutama ketika fitur penting muncul secara lokal, max pooling cenderung lebih efektif karena menjaga respons fitur yang paling menonjol.

Kesimpulan:

Max pooling lebih efektif untuk dataset ini karena mempertahankan fitur visual yang paling menonjol.

E. Perbandingan Forward Propagation Scratch Shared vs Keras

Model terbaik `cnn_2conv_f32-64_k5-5_maxpool` dievaluasi pada test split.

Backend	Test Samples	Macro-F1	Waktu Eksekusi	Agreement dengan Keras Prefix
Keras	30000	0.6323	8.65 s	
Scratch Shared	64	0.0963	0.73 s	1.0000

Disclaimer subset: evaluasi Keras dilakukan pada seluruh test split sebanyak 3000 sampel, sedangkan evaluasi scratch shared hanya dilakukan pada subset 64 sampel. Pembatasan ini dilakukan karena implementasi scratch menggunakan NumPy dan sliding window eksplisit, sehingga proses forward convolution jauh lebih mahal dibanding TensorFlow/Keras yang sudah teroptimasi. Karena jumlah sampelnya berbeda, macro F1 scratch shared sebesar 0.0963 tidak boleh dibandingkan secara langsung dengan macro F1 Keras sebesar 0.6323 sebagai ukuran performa akhir.

Indikator yang paling penting untuk memvalidasi forward propagation scratch adalah nilai agreement dengan Keras pada subset yang sama. Nilai agreement 1.0000 menunjukkan bahwa pada 64 sampel prefix yang dievaluasi, prediksi scratch shared sama dengan prediksi Keras. Artinya, implementasi forward propagation dari scratch sudah konsisten secara logika dengan model Keras untuk arsitektur shared parameter.

Kesimpulan:

Forward propagation scratch shared berhasil mereplikasi prediksi Keras pada subset yang sama. Perbedaan macro F1 utama berasal dari perbedaan jumlah sampel evaluasi, bukan dari perbedaan logika forward propagation.

F. Perbandingan Shared vs Non-Shared Parameter

Evaluasi non-shared dilakukan dengan mengganti semua Conv2D menjadi LocallyConnected2D. Bobot Conv2D yang sudah dilatih direplikasi untuk setiap posisi output.

Model	Macro-F1	Samples	Waktu	Jumlah Parameter	Agreement dengan Keras Prefix
Scratch Shared	0.0963	64	0.73 s	62.790	1.0000
Scratch Non-Shared	0.0963	64	0.78 s	140.534.662	1.0000

Rasio Parameter:

$$\text{Nonshared} / \text{Shared} = 2238.17x$$

Pada subset evaluasi, shared dan non-shared menghasilkan macro F1 yang sama, yaitu 0.0963, serta agreement 1.0000 terhadap prediksi Keras. Hasil yang sama ini wajar karena bobot LocallyConnected2D non-shared dibuat dengan cara mereplikasi bobot Conv2D ke setiap posisi output. Dengan kata lain, walaupun struktur parameternya berubah menjadi non-shared, nilai bobot awalnya masih meniru pola filter shared dari Conv2D.

Perbedaan paling besar terlihat pada jumlah parameter. Model shared hanya memiliki 62.790 parameter, sedangkan model non-shared memiliki 140.534.662 parameter. Rasio ini mencapai 2238.17 kali lebih besar. Non-shared parameter tidak memakai asumsi translational invariance, sehingga pola visual yang sama di lokasi berbeda membutuhkan bobot yang berbeda pula. Hal ini membuat model sangat boros parameter. Sebaliknya, Conv2D shared menggunakan filter yang sama di seluruh posisi spasial, sehingga jauh lebih efisien dan lebih sesuai untuk image classification, karena objek atau tekstur penting dapat muncul di berbagai lokasi gambar.

Kesimpulan:

Parameter sharing pada Conv2D memberikan efisiensi yang sangat besar. Dalam eksperimen ini, non-shared parameter membutuhkan lebih dari 2200 kali parameter dibanding shared, tetapi tidak memberi peningkatan prediksi karena bobotnya hanya hasil replikasi. Ini menunjukkan Conv2D shared lebih cocok untuk image classification karena pola visual dapat muncul di berbagai lokasi gambar.

G. Analisis Perbedaan Hasil

Hasil CNN dapat dijelaskan dari kombinasi kapasitas representasi dan efisiensi parameter. Model dengan 2 conv layer lebih baik daripada 1 conv layer karena mampu membangun representasi visual secara hierarkis, mulai dari fitur sederhana seperti edge dan tekstur hingga pola yang lebih kompleks. Kombinasi filter 32-64 juga lebih unggul karena kapasitasnya lebih besar dibanding 16-32, sehingga model dapat mempelajari lebih banyak variasi fitur dari enam kelas gambar.

Ukuran kernel 5-5 memberikan hasil lebih baik daripada 3-3 karena receptive field yang lebih luas membantu model menangkap konteks lokal yang lebih besar. Selain itu, max pooling lebih unggul daripada average pooling karena mempertahankan aktivasi fitur yang paling kuat dan diskriminatif. Dari sisi implementasi scratch, hasil agreement 1.0000 terhadap Keras pada subset evaluasi menunjukkan bahwa forward propagation shared sudah valid. Sementara itu, non-shared parameter tidak memberi keuntungan prediksi pada setup ini, tetapi membuat jumlah parameter meningkat sangat besar.

H. Kesimpulan CNN

Eksperimen CNN sudah memenuhi kebutuhan evaluasi karena 16 variasi hyperparameter telah dijalankan, macro F1 digunakan sebagai metrik utama, dan grafik training/validation loss tersedia untuk setiap model. Dari seluruh variasi, model terbaik adalah `cnn_2conv_f32-64_k5-5_maxpool` dengan validation macro F1 sebesar 0.6638 dan test macro F1 Keras sebesar 0.6323 pada 3000 sampel.

Implementasi forward propagation scratch shared juga sudah tervalidasi melalui agreement 1.0000 dengan Keras pada subset 64 sampel. Untuk perbandingan shared dan non-shared, `LocallyConnected2D` menghasilkan jumlah parameter 2238.17 kali lebih besar daripada `Conv2D shared`, tetapi tidak memberikan peningkatan prediksi pada setup evaluasi ini. Secara keseluruhan, hasil CNN menunjukkan bahwa arsitektur dengan 2 convolution layer, filter lebih besar, kernel 5, dan max pooling memberikan performa terbaik. Parameter sharing terbukti sangat penting karena membuat model jauh lebih efisien tanpa kehilangan kemampuan mengenali pola visual yang muncul di berbagai lokasi gambar.

2. RNN

Komponen yang digunakan:

Komponen	Nilai
Dataset	Flickr8k
Train captions	30.000
Validation captions	5.000

Test captions	5.000
Feature extractor	InceptionV3 frozen
Feature dimension	2048
Vocabulary Size	7464
Max model input length	38
Embedding dimension	4
Hidden units	2 dan 4
Recurrent layers	1,2,3
Decoder	SimpleRNN dan LSTM
Injection	Pre-inject
Optimizer	Adam
Loss	Sparse Categorical Crossentropy
Epoch	10
Batch size	1024

Jumlah model utama:

2 decoder x 3 variasi layer x 2 variasi hidden state = 12 model

Metrik Evaluasi:

1. BLEU-4

BLEU mengukur kecocokan n-gram antara caption hasil prediksi dan caption referensi. Pada BLEU-4, n-gram yang diperhitungkan sampai 4-gram.

Secara umum:

$$BLEU = BP * \exp\left(\sum_{n=1}^N \frac{1}{N} \log(p_n)\right)$$

dengan:

Simbol	Makna
BP	brevity penalty untuk menghukum caption yang terlalu pendek

pn	precision n-gram ke-n
N	maksimum n-gram, pada eksperimen ini N = 4

2. METEOR

METEOR menghitung kecocokan unigram dengan mempertimbangkan precision, recall, dan penalty urutan/chunk.

Secara umum:

$$METEOR = (1 - penalty) * F_{mean}$$

Metrik ini biasanya lebih sensitif terhadap kecocokan kata individual dibanding BLEU-4.

Ringkasan Hasil Terbaik

Model	Backend	Max Caption Length	BLEU-4	METEOR	Waktu Total	Detik/Gambar
lstm_layers 1_hidden4	Keras	10	0.1405	0.2299	6.67 s	0.00667

Hasil terbaik per decoder:

Decoder	Model	Backend	Max Length	BLEU-4	METEOR	Waktu Total
LSTM	lstm_layers 1_hidden4	Keras	10	0.1405	0.2299	6.67 s
RNN	rnn_layers1 _hidden2	Keras	10	0.1405	0.2299	10.46 s

Interpretasi awal:

- LSTM dan RNN sama-sama dapat mencapai BLEU-4 terbaik 0.1405 pada konfigurasi ringan ini.
- LSTM terbaik lebih cepat dibanding RNN terbaik pada evaluasi Keras.
- Nilai metrik masih rendah karena model sengaja dibuat sangat kecil (hidden_units 2 dan 4, embed_dim 16).

A. Variasi Jumlah Layer dan Hidden State

Tabel berikut membandingkan BLEU-4, METEOR, dan waktu eksekusi untuk semua variasi pada backend Keras dengan max_caption_length = 10.

Hasil SimpleRNN

Layer	Hidden	Model	BLEU-4	METEOR	Waktu Eksekusi
1	2	rnn_layers1_hidden2	0.1405	0.2299	10.46 s
1	4	rnn_layers1_hidden4	0.0000	0.0000	1.25 s
2	2	rnn_layers2_hidden2	0.0000	0.0000	1.42 s
2	4	rnn_layers2_hidden4	0.1405	0.2299	12.54 s
3	2	rnn_layers3_hidden2	0.1405	0.2299	17.14 s
3	4	rnn_layers3_hidden4	0.1405	0.2299	35.44 s

Hasil LSTM

Layer	Hidden	Model	BLEU-4	METEOR	Waktu Eksekusi
1	2	lstm_layers1_hidden2	0.0000	0.0000	1.74 s
1	4	lstm_layers1_hidden4	0.1405	0.2299	6.67 s
2	2	lstm_layers2_hidden2	0.1405	0.2299	7.18 s
2	4	lstm_layers2_hidden4	0.1405	0.2299	9.90 s
3	2	lstm_layers3_hidden2	0.1320	0.1498	9.69 s
3	4	lstm_layers3_hidden4	0.1405	0.2299	8.67 s

Pada konfigurasi yang sangat kecil ini, hidden state yang lebih besar tidak selalu menghasilkan skor yang lebih tinggi. Contohnya, rnn_layers1_hidden2 mencapai BLEU-4 sebesar 0.1405, sedangkan rnn_layers1_hidden4 justru mendapat 0.0000. Hal ini menunjukkan bahwa ketika kapasitas model terlalu kecil, perubahan hidden state dari 2 ke 4 belum cukup stabil untuk

meningkatkan kualitas caption. Model dapat jatuh ke pola prediksi yang terlalu umum atau repetitif, sehingga skor BLEU-4 dan METEOR menjadi rendah.

Pada LSTM, variasi hidden 4 terlihat lebih stabil dibanding hidden 2 karena 4 dari 6 variasi LSTM mencapai BLEU-4 sekitar 0.1405. Penambahan jumlah recurrent layer cenderung meningkatkan waktu eksekusi, terutama pada RNN. Sebagai contoh, rnn_layers3_hidden4 membutuhkan 35.44 detik, jauh lebih lama dibanding rnn_layers1_hidden4 yang hanya membutuhkan 1.25 detik. Pada LSTM, kenaikan waktu tidak selalu linear pada hasil run ini, tetapi secara teori model dengan 2-3 layer tetap lebih berat daripada model 1 layer karena setiap timestep harus melewati lebih banyak operasi recurrent.

Kesimpulan:

Pada konfigurasi sangat kecil, efek variasi layer dan hidden state terlihat tidak stabil terhadap kualitas caption. Namun dari sisi efisiensi, semakin banyak layer dan semakin besar hidden state cenderung meningkatkan biaya komputasi. LSTM dengan hidden 4 menunjukkan hasil paling konsisten dibanding variasi lain.

B. Training Loss dan Validation Loss

Grafik training loss dan validation loss sudah dibuat dan tersimpan pada:

artifacts/plots/captioning/history/

Terdapat 12 plot loss untuk eksperimen utama RNN/LSTM, masing-masing satu untuk setiap kombinasi decoder, jumlah layer, dan hidden state. Selain itu, versi plot history umum juga tersedia di `artifacts/plots/captioning/`.

Setiap model dilatih selama 10 epoch. Ringkasan kurva loss dari file *_history.json:

Model	Epoch	Train Loss Awal	Train Loss Akhir	Val Loss Awal	Val Loss Akhir	Best Val Loss
lstm_layers1_hidden2	10	2.7423	2.0782	2.7013	2.0640	2.0640
lstm_layers1_hidden4	10	2.7407	1.8144	2.6936	1.8111	1.8111
lstm_layers2_hidden2	10	2.7460	2.0723	2.7159	2.0571	2.0571
lstm_layers2_hidden4	10	2.7403	1.8205	2.6898	1.8165	1.8165
lstm_layers3_hidden2	10	2.7530	2.0409	2.7314	2.0260	2.0260
lstm_layers3_hidden4	10	2.7447	1.8047	2.6997	1.8020	1.8020

rnn_layers1_hidden2	10	2.7513	2.0694	2.7305	2.0545	2.0545
rnn_layers1_hidden4	10	2.7255	1.8426	2.6581	1.8368	1.8368
rnn_layers2_hidden2	10	2.7421	2.0829	2.7026	2.0683	2.0683
rnn_layers2_hidden4	10	2.7115	1.8598	2.6402	1.8523	1.8523
rnn_layers3_hidden2	10	2.7345	2.1015	2.6898	2.0861	2.0861
rnn_layers3_hidden4	10	2.7178	1.8564	2.6492	1.8493	1.8493

Jika data history tersebut diplot sebagai grafik training loss dan validation loss, seluruh model menunjukkan tren loss yang menurun dari epoch awal ke epoch akhir. Hal ini menandakan bahwa proses optimisasi berjalan dan model berhasil menyesuaikan bobot terhadap data training. Secara umum, hidden 4 menghasilkan loss akhir yang lebih rendah daripada hidden 2 pada hampir semua pasangan model, sehingga dari sisi token-level prediction, kapasitas hidden state yang sedikit lebih besar membantu model belajar distribusi kata dengan lebih baik.

Best validation loss paling rendah pada hasil ini diperoleh oleh lstm_layers3_hidden4 dengan val_loss = 1.8020. Namun, penurunan loss tidak selalu berbanding lurus dengan peningkatan BLEU-4. Hal ini terjadi karena loss training dihitung pada level token dengan teacher forcing, sedangkan BLEU-4 mengevaluasi caption hasil generation secara penuh. Model dapat memiliki loss token-level yang membaik, tetapi saat inference tetap menghasilkan caption yang terlalu generic atau repetitif, sehingga skor BLEU-4 tidak naik secara sebanding.

Kesimpulan training curve:

Secara optimisasi, model belajar karena loss turun stabil. Namun karena kapasitas model dibuat sangat kecil, penurunan loss belum cukup untuk menghasilkan caption yang semantik dan variatif.

C. Perbandingan Keras vs Scratch

Rata-rata seluruh evaluasi utama:

Backend	Jumlah Row	Avg BLEU-4	Avg METEOR	Avg Seconds/Image
Keras	36	0.0612	0.1476	0.0242
Scratch	36	0.0612	0.1476	0.0544

Ringkasan eksplisit per decoder pada `max\_caption\_length = 10`:

Decoder	Backend	Jumlah Model	Avg-BLE U-4	Avg METEOR	Avg Waktu Evaluasi	Avg Detik/Gambar
RNN	Keras	6	0.0937	0.1533	13.04 s	0.0130
RNN	Scratch	6	0.0937	0.1533	26.32 s	0.0263
LSTM	Keras	6	0.1157	0.1783	7.31 s	0.0073
LSTM	Scratch	6	0.1157	0.1783	21.11 s	0.0211

Skor Keras dan Scratch sama pada semua model yang dibandingkan. Hasil ini menunjukkan bahwa implementasi forward propagation scratch berhasil membaca bobot hasil training Keras dan menghasilkan prediksi yang konsisten. Karena bobot, arsitektur, dan alur decoding yang digunakan sama, tidak muncul perbedaan skor berarti antara backend Keras dan Scratch pada evaluasi utama.

Perbedaan utama terdapat pada waktu eksekusi. Pada RNN, Scratch membutuhkan rata-rata 26.32 detik, sedangkan Keras 13.04 detik. Pada LSTM, Scratch membutuhkan rata-rata 21.11 detik, sedangkan Keras 7.31 detik. Scratch lebih lambat karena implementasinya menggunakan NumPy dan loop Python, sedangkan Keras berjalan di atas TensorFlow yang memiliki operasi tensor teroptimasi, graph execution, dan kemungkinan optimasi backend. Dengan demikian, scratch valid untuk menunjukkan pemahaman forward propagation, tetapi Keras tetap lebih efisien untuk inference skala besar.

Kesimpulan Keras vs Scratch:

Implementasi scratch valid secara fungsi karena skor sama dengan Keras, tetapi lebih lambat karena tidak memakai graph execution dan optimasi backend TensorFlow.

D. Perbandingan RNN vs LSTM

Pada evaluasi Keras dengan max_caption_length = 10, rata-rata skor per decoder:

Decoder	Jumlah Model	Avg BLEU-4	Avg METEOR	Avg Waktu
RNN	6	0.0937	0.1533	13.04
LSTM	6	0.1157	0.1783	7.31

Pada evaluasi Keras dengan `max_caption_length = 10`, LSTM memiliki rata-rata BLEU-4 dan METEOR yang lebih tinggi daripada RNN. Rata-rata BLEU-4 LSTM adalah 0.1157, sedangkan RNN berada pada 0.0937. Rata-rata METEOR LSTM juga lebih tinggi, yaitu 0.1783 dibanding 0.1533 pada RNN. Dari sisi waktu, LSTM juga lebih cepat secara rata-rata pada hasil run ini, terutama karena variasi RNN 3 layer hidden 4 membutuhkan waktu sangat besar.

Secara teori, keunggulan LSTM berasal dari mekanisme gate di mana mekanisme cell state `c_t` membantu LSTM mempertahankan informasi jangka panjang. Pada captioning, hal ini penting karena decoder perlu mengingat konteks kata sebelumnya saat menghasilkan token berikutnya.

Karena tidak memiliki gate dan cell state, SimpleRNN lebih rentan terhadap vanishing gradient ketika sequence panjang. Pada konfigurasi kecil ini, perbedaannya tidak selalu dramatis karena model sangat terbatas, tetapi rata-rata LSTM tetap lebih unggul.

Kesimpulan RNN vs LSTM:

LSTM lebih stabil dan sedikit lebih baik daripada RNN pada eksperimen ini. Keunggulan LSTM sesuai teori karena LSTM memiliki cell state dan gate untuk menjaga informasi jangka panjang, sehingga lebih cocok untuk caption generation dibanding SimpleRNN.

E. Perbandingan Pre-inject vs Init-inject

Selain pre-inject, eksperimen bonus init-inject juga dijalankan. Pada pre-inject, fitur gambar diberikan sebagai input awal ke decoder. Pada init-inject, fitur gambar digunakan untuk menginisialisasi state awal recurrent layer. Perbandingan berikut menggunakan backend Keras pada `max_caption_length = 10`.

Skema	Decoder	Jumlah Model	Avg BLEU-4	Avg METEOR	Model Terbaik
Pre-Inject	RNN	6	0.0937	0.1533	rnn_layers1_hidden2
Init-Inject	RNN	6	0.0661	0.0870	init_rnn_layers1_hidden4
Pre-Inject	LSTM	6	0.1157	0.1783	lstm_layers1_hidden4
Init-Inject	LSTM	6	0.1171	0.1916	init_lstm_layers1_hidden4

Init-inject tidak memberikan peningkatan yang merata untuk semua decoder. Pada SimpleRNN, rata-rata BLEU-4 dan METEOR init-inject lebih rendah dibanding pre-inject. Hal ini menunjukkan bahwa menjadikan fitur gambar sebagai initial hidden state belum cukup stabil

untuk RNN sederhana dengan kapasitas hidden state yang sangat kecil. Informasi visual yang dimasukkan ke state awal mudah hilang sepanjang proses decoding.

Pada LSTM, init-inject sedikit lebih baik daripada pre-inject secara rata-rata. Rata-rata BLEU-4 naik dari 0.1157 menjadi 0.1171, dan METEOR naik dari 0.1783 menjadi 0.1916. Mekanisme cell state pada LSTM kemungkinan membantu mempertahankan informasi visual awal lebih baik dibanding SimpleRNN. Namun, selisihnya masih kecil dan kualitas caption tetap terbatas karena kapasitas model sangat rendah.

Kesimpulan Pre-inject vs Init-inject:

Init-inject bisa dikatakan lebih baik pada LSTM dibanding SimpleRNN, tetapi pada konfigurasi kecil ini belum cukup untuk memperbaiki kualitas caption secara signifikan. Skema injeksi fitur memengaruhi hasil, tetapi kapasitas decoder tetap menjadi faktor pembatas utama.

F. Pengaruh Panjang Maksimum Caption

Evaluasi dilakukan dengan tiga variasi panjang maksimum caption: 10, 20, dan 38. Model tetap menerima input sequence panjang 38, tetapi proses generation dibatasi sesuai nilai maksimum. Rata-rata backend Keras:

Max Caption Length	Avg BLEU-4	Avg METEOR	Avg Waktu
10	0.1047	0.1658	10.18 s
20	0.0518	0.1494	22.30 s
38	0.0270	0.1276	40.10 s

Nilai `max_caption_length = 10` menghasilkan rata-rata BLEU-4 dan METEOR tertinggi dibanding panjang 20 dan 38. Pada panjang 10, rata-rata BLEU-4 adalah 0.1047 dan rata-rata METEOR adalah 0.1658. Ketika panjang maksimum dinaikkan menjadi 20 dan 38, skor justru turun, sementara waktu evaluasi meningkat cukup besar. Hal ini wajar karena decoder harus melakukan lebih banyak timestep ketika caption yang diizinkan semakin panjang.

Pada model kecil ini, caption yang terlalu panjang cenderung menjadi repetitif. Repetisi token membuat precision n-gram turun, sehingga BLEU-4 ikut menurun. Panjang 38 memang merupakan panjang maksimum sequence hasil preprocessing, tetapi nilai tersebut tidak otomatis menjadi pilihan terbaik untuk generation. Untuk model dengan kapasitas kecil, caption yang lebih pendek justru memberi trade-off lebih baik antara skor dan waktu inference.

Kesimpulan Pengaruh Panjang Maksimum Caption:

Untuk model kecil yang digunakan, caption pendek lebih efektif. `max_caption_length = 10` memberikan trade-off terbaik antara skor dan waktu eksekusi.

G. Beam Search vs Greedy

Beam search dievaluasi sebagai bonus dengan beam_width = 3 dan beam_width = 5, max_caption_length = 20, dan limit_images = 50. Rata-rata hasil:

Search	Jumlah Row	Avg BLEU-4	Avg METEOR	Avg Seconds/Image
Greedy	12	0.0516	0.1490	0.0240
Beam Search k=3	12	0.0665	0.1543	0.9779
Beam Search k=5	12	0.0603	0.1539	1.0007

Hasil terbaik beam/greedy pada evaluasi bonus:

Model	Search	Avg BLEU-4	Avg METEOR	Waktu Total	Avg Seconds/Image
lstm_layers1_hidden2	Beam k=3	0.1785	0.0632	12.23 s	0.2446
lstm_layers1_hidden2	Beam k=5	0.1043	0.0591	15.56s	0.3113
lstm_layers1_hidden4	Greedy	0.0691	0.2049	1.49s	0.0297
lstm_layers1_hidden4	Beam k=3	0.0691	0.2049	36.96	0.7392

Beam search k=3 menaikkan rata-rata BLEU-4 dari 0.0516 menjadi 0.0665 pada subset 50 gambar. METEOR juga naik sedikit dari 0.1490 menjadi 0.1543. Kenaikan ini menunjukkan bahwa mempertahankan beberapa kandidat caption pada setiap timestep dapat membantu decoder menemukan urutan kata yang lebih baik dibanding greedy decoding yang selalu mengambil token terbaik secara lokal.

Beam search k=5 tidak selalu lebih baik daripada k=3. Pada model pre-inject, k=5 menghasilkan BLEU-4 rata-rata 0.0603, lebih rendah dari k=3 sebesar 0.0665, tetapi tetap di atas greedy. Waktu k=5 juga sedikit lebih mahal, yaitu sekitar 1.0007 detik per gambar dibanding 0.9779 detik per gambar pada k=3. Pada model yang sangat kecil, beam yang lebih besar dapat mengeksplorasi lebih banyak kandidat, tetapi kandidat tambahan tersebut tetap berasal dari distribusi model yang underfit dan bias ke token umum.

Kesimpulan Beam Search vs Greedy:

Beam search membantu eksplorasi kandidat caption dan dapat menaikkan BLEU-4, tetapi biayanya jauh lebih mahal. Pada eksperimen ini, k=3 menjadi pilihan yang lebih baik daripada k=5 karena memberi peningkatan metrik lebih besar dengan waktu yang lebih rendah. Pada model yang terlalu kecil, beam search belum tentu memperbaiki kualitas kualitatif karena distribusi probabilitas model sudah bias ke token umum/repetitif.

H. Qualitative Analysis

Catatan: karena model sengaja dibuat sangat kecil, prediksi yang muncul sangat repetitif, terutama token a. Hal ini menunjukkan underfitting dan kapasitas representasi yang rendah.

Kategori	Image	Image ID	LSTM Caption	RNN Caption	GT Singkat	Skor Tertinggi
Tinggi		389643437_9a9830a3ba	a a a a a a a a a a	a a a a a a a a a a	a girl in a white cap eats a slice of fruit next to a girl in a blue cap who is eating chocolate	0.1659
Tinggi		482353373_03a9d5e8bc	a a a a a a a a a a	a a a a a a a a a a	a girl in a petterned shirt is talking to a guy with a hat in a billiards room	0.1659
Tinggi		517094985_4b9e926936	a a a a a a a a a a	a a a a a a a a a a	a child is riding a unicycle along a riverside path next to a woman who is walking a dog	0.1659

Sedang		439492931_a96d590e40	a a a a a a a a a a	a a a a a a a a a a	a cluster of four brown dogs play in a field of brown grass	0.1395
Sedang		440190907_bf8b7ba8ef	a a a a a a a a a a	a a a a a a a a a a	a group of people walk through a shopping mall	0.1395
Sedang		441212506_fcc321ac28	a a a a a a a a a a	a a a a a a a a a a	the black dog runs with a ball with two smaller dogs behind it	0.1395
Rendah		783353797_fdf91bdf4c	a a a a a a a a a a	a a a a a a a a a a	a girl is on the sidewalk looking at a white van in the street	0.0935
Rendah		467858872_f3431df682	a a a a a a a a a a	a a a a a a a a a a	two dogs are bending over and looking down into a ditch to investigate	0.0934
Rendah		756909515_a416161656	a a a a a a a a a a	a a a a a a a a a a	three dogs are playing in the water	0.0868

RNN dan LSTM sering menghasilkan caption yang sama pada contoh kualitatif di atas. Prediksi repetitif seperti a a a ... atau on on ... muncul karena model terlalu kecil dan distribusi token training sangat bias ke kata umum. Skor BLEU/METEOR masih dapat bernilai tidak nol ketika token umum seperti a muncul di ground truth, tetapi secara semantik caption tersebut belum menggambarkan isi gambar dengan baik.

Model juga belum mampu menangkap objek spesifik seperti dog, girl, soccer, pelican, atau skateboard. Hal ini menunjukkan underfitting: model mempelajari pola kata yang sering muncul, tetapi belum cukup kuat untuk mengaitkan fitur visual 2048 dimensi dari CNN encoder dengan konsep objek dan aksi dalam caption. Dengan hidden state hanya 2 dan 4, kapasitas representasi recurrent layer terlalu kecil untuk menyimpan konteks visual dan konteks bahasa secara kaya.

I. Analisis kualitatif RNN vs LSTM:

Pada konfigurasi kecil ini, perbedaan kualitatif RNN dan LSTM belum terlalu terlihat karena keduanya sama-sama underfit. Secara teori, LSTM lebih unggul untuk sequence panjang karena cell state dapat membantu mengurangi vanishing gradient dan mempertahankan informasi jangka panjang. Namun, dengan hidden state 2 dan 4, kapasitas LSTM juga terlalu kecil untuk menunjukkan keunggulan tersebut secara jelas. Jika hidden state diperbesar, misalnya ke 128 atau 512, LSTM diperkirakan akan lebih unggul secara kualitatif karena dapat mempertahankan konteks caption lebih baik daripada SimpleRNN.

BAB 3

Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan eksperimen yang telah dilakukan pada arsitektur Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN/LSTM), dapat ditarik beberapa kesimpulan utama. Pada tugas klasifikasi gambar, arsitektur model yang lebih dalam dan lebar terbukti menghasilkan performa terbaik. Menambah jumlah layer konvolusi dan menggunakan kombinasi filter yang lebih besar memberikan kapasitas representasi yang cukup bagi model untuk mengekstraksi fitur visual hierarkis secara optimal pada dataset Intel Image Classification.

Variasi ukuran kernel dan jenis pooling memberikan dampak yang signifikan terhadap kualitas prediksi akhir. Penggunaan kernel yang lebih besar lebih unggul karena receptive field yang lebih luas mampu menangkap konteks lokal yang lebih representatif. Sementara itu, Max Pooling terbukti secara konsisten lebih efektif dibandingkan Average Pooling, karena metode ini mampu mempertahankan aktivasi fitur yang paling kuat dan diskriminatif (seperti tepi objek atau garis batas).

Dalam perbandingan arsitektur parameter sharing (Conv2D) dengan non-shared parameter (LocallyConnected2D), konsep weight sharing terbukti sangat krusial. Model non-shared memicu pembengkakan jumlah parameter yang sangat drastis tanpa memberikan peningkatan performa prediksi sama sekali. Hal ini menegaskan bahwa asumsi translational invariance pada CNN sangat esensial dan jauh lebih efisien untuk memproses data citra.

Pada tugas image captioning, LSTM secara umum lebih stabil dan sedikit lebih unggul dibanding SimpleRNN, terutama pada evaluasi utama pre-inject. Mekanisme gate dan cell state pada LSTM membantunya mempertahankan informasi sekuensial lebih baik dibanding RNN biasa. Namun, karena konfigurasi decoder dibuat sangat kecil, yaitu hidden state 2 dan 4 serta embedding dimension rendah, seluruh model masih mengalami underfitting. Caption yang dihasilkan cenderung repetitif dan belum mampu merepresentasikan semantik visual secara baik.

Dalam perbandingan dengan library Keras, implementasi forward propagation buatan sendiri (from scratch) menunjukkan hasil evaluasi prediksi yang sepenuhnya identik, membuktikan bahwa logika propagasi matematis telah diimplementasikan dengan presisi yang tepat. Akan tetapi, Keras jauh lebih unggul secara efisiensi waktu eksekusi berkat optimasi graph execution

dan operasi tensor internalnya. Di sisi strategi decoding, penggunaan Beam Search memang mampu meningkatkan skor metrik evaluasi secara marginal dibandingkan Greedy Decoding. Beam Search dengan $k=3$ memberikan hasil lebih baik dibanding $k=5$, sekaligus sedikit lebih efisien. Namun, peningkatan kualitas tersebut harus dibayar dengan waktu inferensi yang jauh lebih besar. Pada model yang underfit, Beam Search juga belum mampu memperbaiki kualitas semantik caption secara signifikan karena distribusi probabilitas model sudah bias terhadap token umum dan repetitif.

Kemudian, eksperimen init-inject sebagai skema injeksi fitur alternatif menunjukkan hasil yang berbeda untuk RNN dan LSTM. Pada RNN, init-inject tidak memperbaiki performa rata-rata dibanding pre-inject. Sebaliknya, pada LSTM, init-inject memberikan sedikit peningkatan rata-rata metrik. Hal ini menunjukkan bahwa LSTM lebih mampu memanfaatkan informasi visual yang diberikan sebagai initial state karena memiliki mekanisme cell state dan gate.

B. Saran

Untuk pengembangan model, eksperimen lanjutan, maupun penyempurnaan implementasi ke depannya, beberapa langkah berikut sangat direkomendasikan:

1. Peningkatan Kapasitas Model Captioning (RNN/LSTM)

Mengingat hasil evaluasi kualitatif menunjukkan indikasi underfitting yang parah (generasi teks yang repetitif), sangat krusial untuk menaikkan kapasitas decoder. Disarankan untuk meningkatkan ukuran hidden state secara signifikan dan memperbesar dimensi embedding. Hal ini akan memberi ruang parameter yang cukup bagi model untuk mempelajari pemetaan kompleks antara vektor fitur citra dengan sekuens kata.

2. Penyesuaian Panjang Maksimum Caption Terhadap Kapasitas Model

Hasil eksperimen menunjukkan bahwa memaksakan sequence length generasi yang terlalu panjang pada model berkapasitas rendah justru memperburuk skor metrik akibat repetisi token. Untuk model eksperimental berskala kecil, mempertahankan batasan panjang generasi di angka yang lebih pendek terbukti menjadi titik optimal (sweet spot) antara efisiensi waktu dan kualitas metrik.

3. Optimasi Eksekusi pada Implementasi From Scratch

Agar perbandingan kecepatan eksekusi dengan library Keras lebih berimbang, disarankan untuk mengimplementasikan teknik vektorisasi yang lebih efisien untuk operasi konvolusi (misalnya menggunakan metode `im2col`) alih-alih menggunakan looping

spasial murni, atau memanfaatkan pustaka komputasi array berbasis GPU untuk mempercepat proses training dan inference.

4. Eksperimen init inject

Eksperimen pre-inject dan init-inject menunjukkan bahwa cara memasukkan fitur visual ke decoder berpengaruh terhadap performa, tetapi dampaknya berbeda antara SimpleRNN dan LSTM. Init-inject belum meningkatkan performa SimpleRNN, namun memberikan sedikit peningkatan pada LSTM. Ke depannya, eksperimen init-inject dapat dicoba dengan kapasitas decoder yang lebih besar, regularisasi yang lebih sesuai, dan durasi training yang lebih panjang agar pengaruh initial state dari fitur gambar dapat dimanfaatkan secara lebih optimal.

5. Strategi decoding dapat dieksplorasi lebih lanjut.

Beam Search $k=3$ lebih efektif daripada $k=5$ pada eksperimen ini, tetapi kualitas caption masih rendah karena model dasarnya underfit. Setelah model diperbagus, beam width yang berbeda dapat diuji ulang untuk melihat apakah beam search benar-benar meningkatkan kualitas semantik, bukan hanya skor n-gram.

6. Gunakan RTX 5090



REFERENSI

- [1] S. Sena, “Pengenalan Deep Learning Part 7 : Convolutional Neural Network (CNN),” Medium, [Online]. Available: <https://medium.com/@samuelsena/pengenalan-deep-learning-part-7-convolutional-neural-network-cnn-b003b477dc94>. [Accessed: May 8, 2026].
- [2] A. Ali, “Recurrent Neural Network & LSTM with Practical Implementation,” Medium, [Online]. Available: <https://medium.com/machine-learning-researcher/recurrent-neural-network-rnn-e6f69db16eba>. [Accessed: May 8, 2026].
- [3] N. Fanelli, “End-to-End Implementation of an Image Caption Generator with CNN and RNN using TensorFlow and Keras,” Medium, [Online]. Available: <https://medium.com/@nicolafan/end-to-end-implementation-of-an-image-caption-generator-with-cnn-and-rnn-using-tensorflow-and-keras-589f8ddea32b>. [Accessed: May 9, 2026].
- [4] GeeksforGeeks, “Image Caption Generator using Deep Learning on Flickr8K dataset,” GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/image-caption-generator-using-deep-learning-on-flickr8k-dataset/>. [Accessed: May 9, 2026].
- [5] P. Solai, “Convolutions and Backpropagations,” Medium, [Online]. Available: <https://pavisj.medium.com/convolutions-and-backpropagations-46026a8f5d2c>. [Accessed: May 12, 2026].
- [6] J. D. Suo, “Back to basics: Deriving back-propagation on simple RNN & LSTM,” Medium, [Online]. Available: <https://medium.com/data-science/back-to-basics-deriving-back-propagation-on-simple-rnn-lstm-feat-aidan-gomez-c7f286ba973d>. [Accessed: May 12, 2026].

LAMPIRAN

Pembagian Tugas:

Implementasi	Penanggung Jawab
Integrasi pipeline training, inference, dan evaluasi dari bagian CNN serta RNN/LSTM, implementasi helper umum (seperti load/save weights, metrik, waktu, dan plotting history training), pelaksanaan dan pengumpulan hasil eksperimen utama (evaluasi macro F1-score CNN, BLEU-4, METEOR, waktu eksekusi Keras vs scratch, serta grafik loss), serta pelaksanaan dan perekapan eksperimen bonus (feature maps, Grad-CAM, pre-inject vs init-inject, beam search, batch inference, dan backward propagation)	13523029
Feature extraction CNN encoder untuk Flickr8k, preprocessing caption, decoder Keras untuk Simple RNN dan LSTM dengan arsitektur pre-inject, variasi eksperimen captioning, forward propagation from scratch, inference caption generation untuk RNN dan LSTM, implementasi bonus RNN/LSTM, dan evaluasi BLEU-4 serta METEOR untuk semua variasi RNN/LSTM	13523033
Implementasi utility image loading untuk dataset Intel Image Classification, pipeline training CNN Keras dengan Conv2D shared parameter, variasi eksperimen CNN, forward propagation CNN from scratch, implementasi bonus CNN (visualisasi intermediate feature maps, Grad-CAM, batch inference, dan backward propagation), evaluasi CNN menggunakan macro F1-score, serta analisis perbandingan (Keras vs scratch, shared vs non-shared parameter, dan pengaruh hyperparameter CNN)	13523040

Link GitHub: <https://github.com/KennethhPoenadi/ML-KPEZ>

Surat Pernyataan Penggunaan AI:  Surat Pernyataan Penggunaan AI